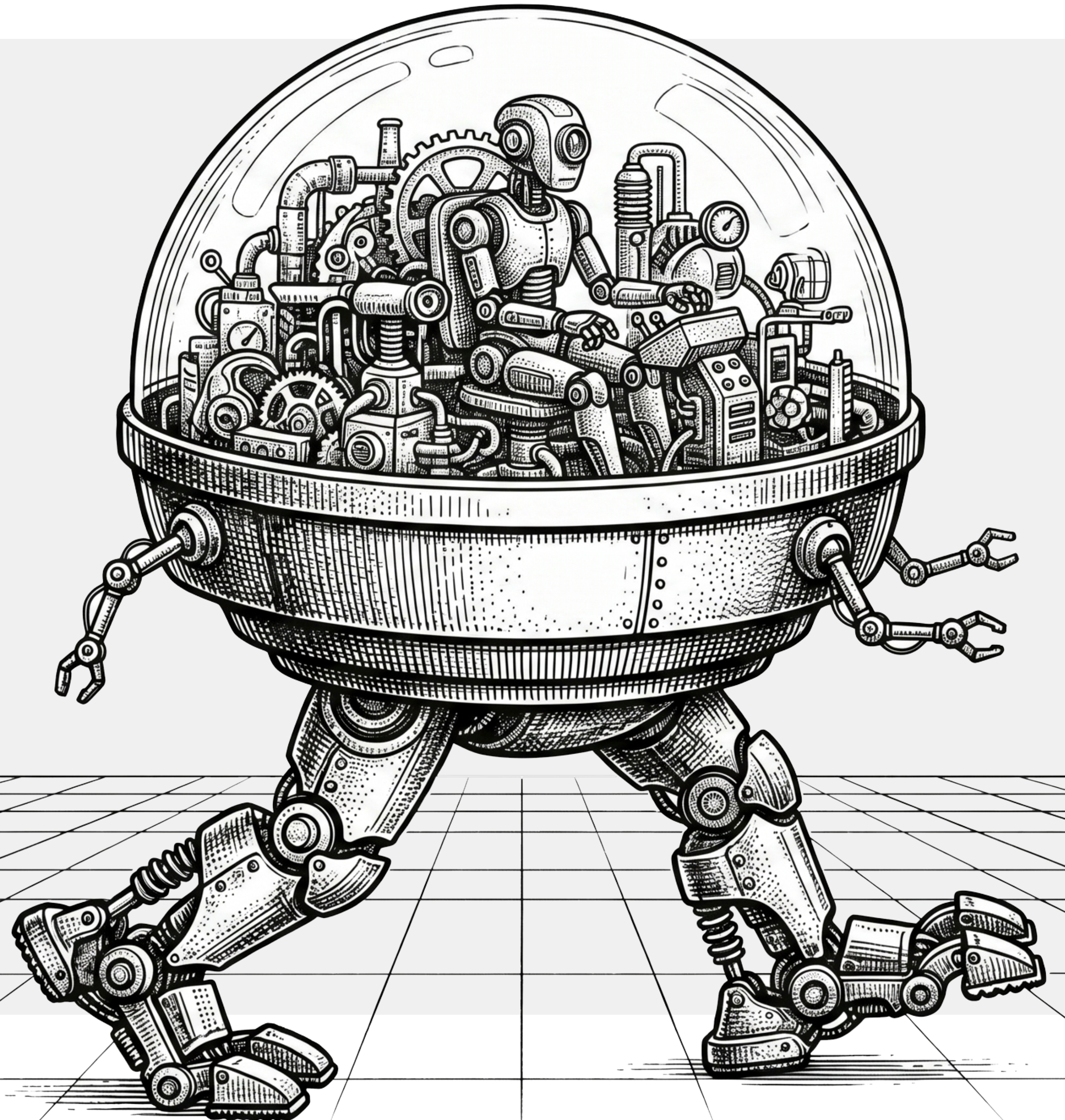


Introducción al Aprendizaje Automático

Felipe Ramírez Herrera



Laboratorio 1: Modelos

Iris Dataset.

- RandomForest
- KNN y SVM
- Normalización y MLP
- DNN con Pytorch***

Comparemos los modelos.



Ejercicio Práctico(10 min)

Adapte lo visto anteriormente para el conjunto de datos de ejemplo Wine.

```
import pandas as pd  
import matplotlib.pyplot as plt  
import seaborn as sns  
from sklearn.datasets import load_wine
```

```
wine = load_wine()
```



Generalización

La generalización es la capacidad de un modelo para realizar predicciones precisas y comportarse adecuadamente frente a datos nuevos y nunca antes vistos (es decir, datos que no formaron parte de su proceso de entrenamiento).

Objetivo:

Cuando un algoritmo aprende, evaluamos su éxito utilizando dos métricas principales:

Error de entrenamiento (Training Error):

Mide los errores cometidos en los datos específicos utilizados para enseñar al modelo.

Error de generalización o prueba (Test

Error): Mide los errores cometidos en datos no vistos previamente.

Desafíos comunes:

Sobreajuste (Overfitting): Funciona a la perfección en el entrenamiento, pero falla con datos nuevos.

Subajuste (Underfitting): Tiene un rendimiento deficiente tanto en los datos de entrenamiento como en los de prueba.

El objetivo es encontrar el punto de equilibrio matemático donde la brecha entre el error de entrenamiento y el de prueba se minimice, maximizando así la precisión predictiva.

Regularización

Técnicas que penalizan los modelos excesivamente complejos, fomentando pesos de parámetros más simples.

Abandono (Dropout)

Una técnica en la que se “apagan” neuronas al azar durante el entrenamiento para evitar que la red neuronal dependa demasiado de rutas o detalles específicos.

Aumento de datos (Data Augmentation)

Agregar ligeras variaciones, ruido o transformaciones a los datos de entrenamiento, haciendo que el modelo sea más robusto ante anomalías del mundo real.

Parada temprana (Early Stopping)

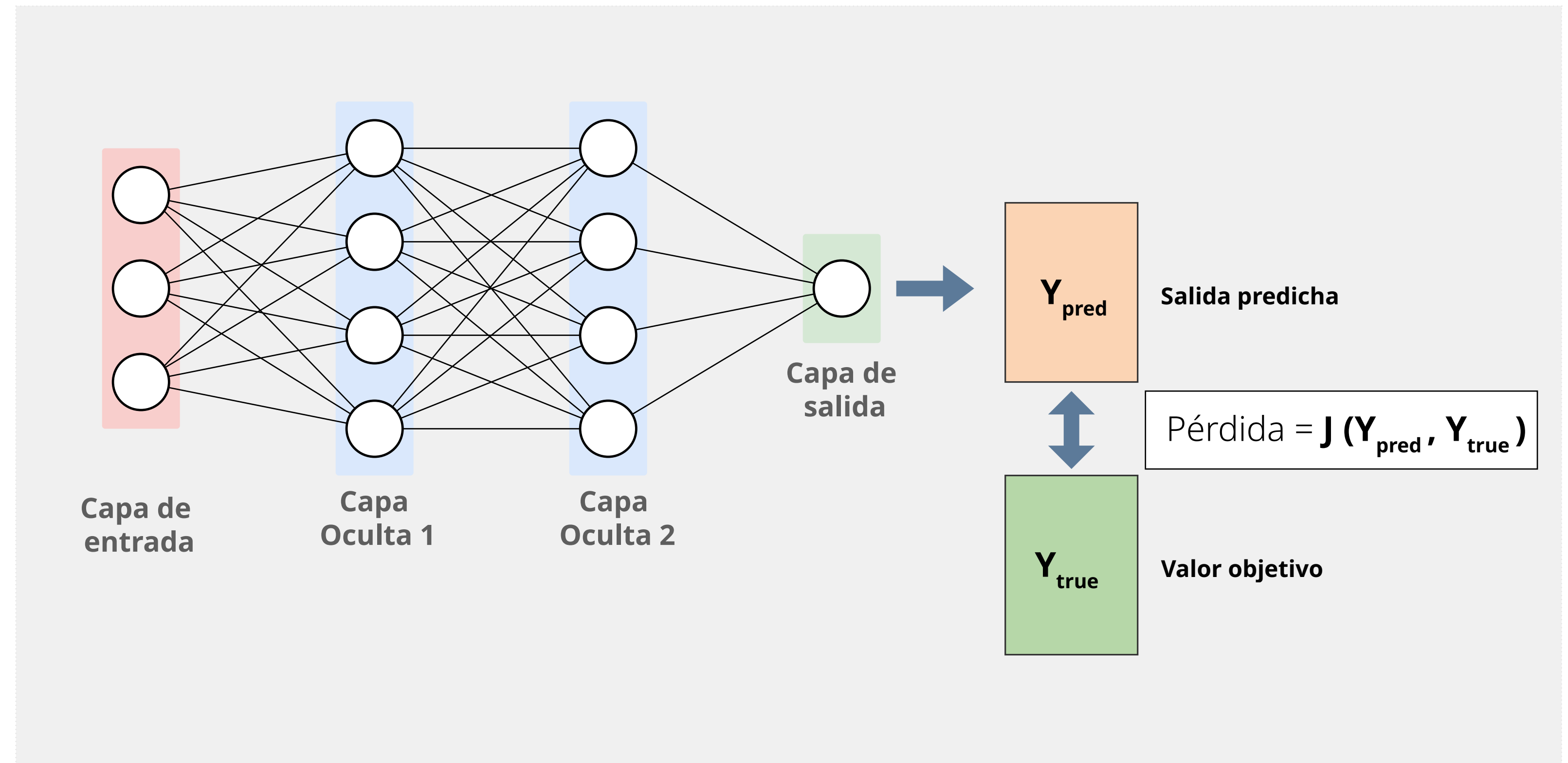
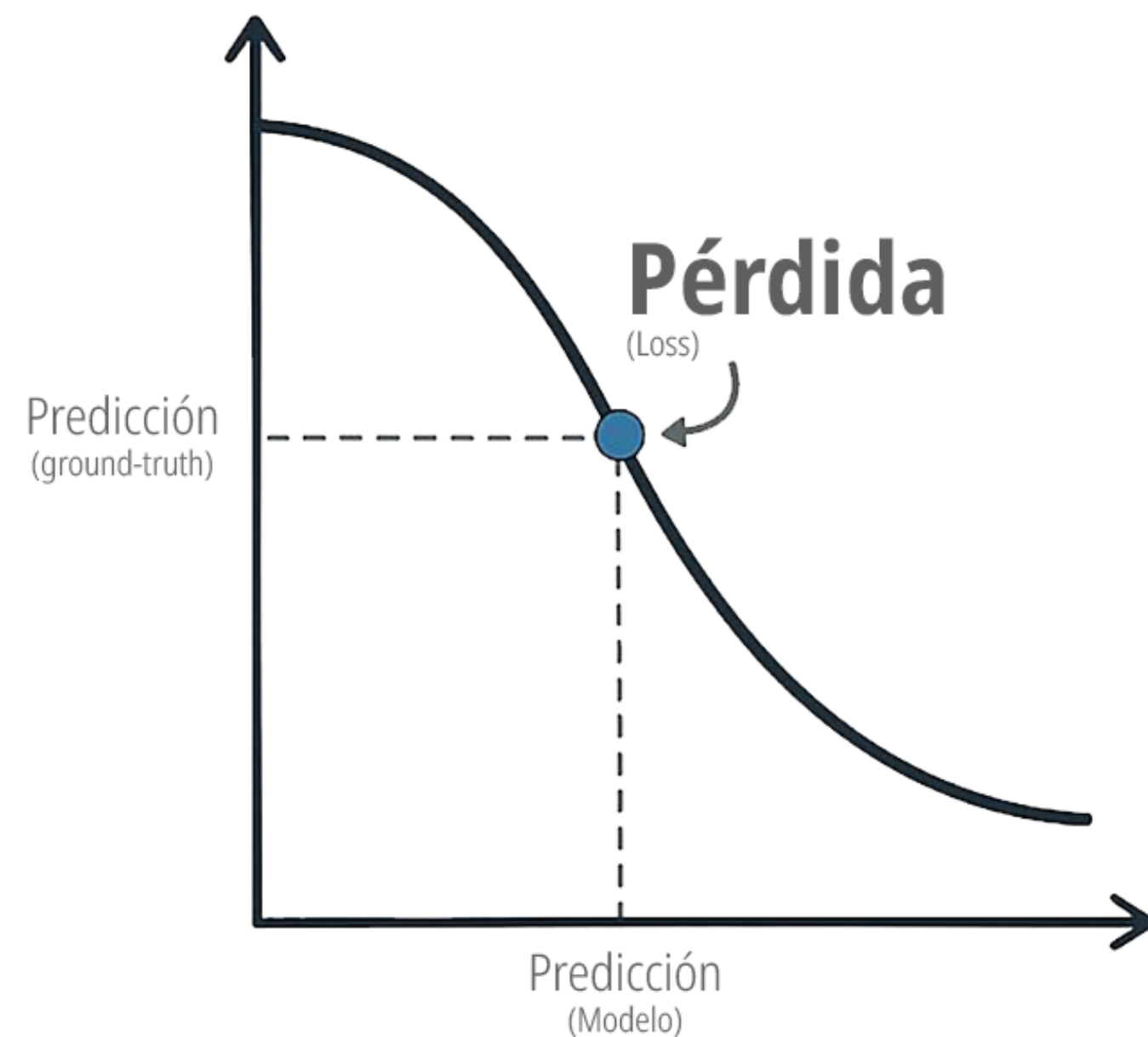
Detener el proceso de entrenamiento tan pronto como el error en un conjunto de datos de validación comienza a aumentar, evitando que el modelo se optimice en exceso para los datos de entrenamiento.

Validación cruzada (Cross-Validation)

Dividir los datos en múltiples subconjuntos para probar repetidamente cómo se comporta el modelo con entradas no vistas antes de su implementación final.

Función de pérdida (loss)

Una función de pérdida (loss function) es una fórmula matemática que cuantifica la diferencia entre la salida predicha por un modelo y el valor real o “verdad fundamental” (ground truth).

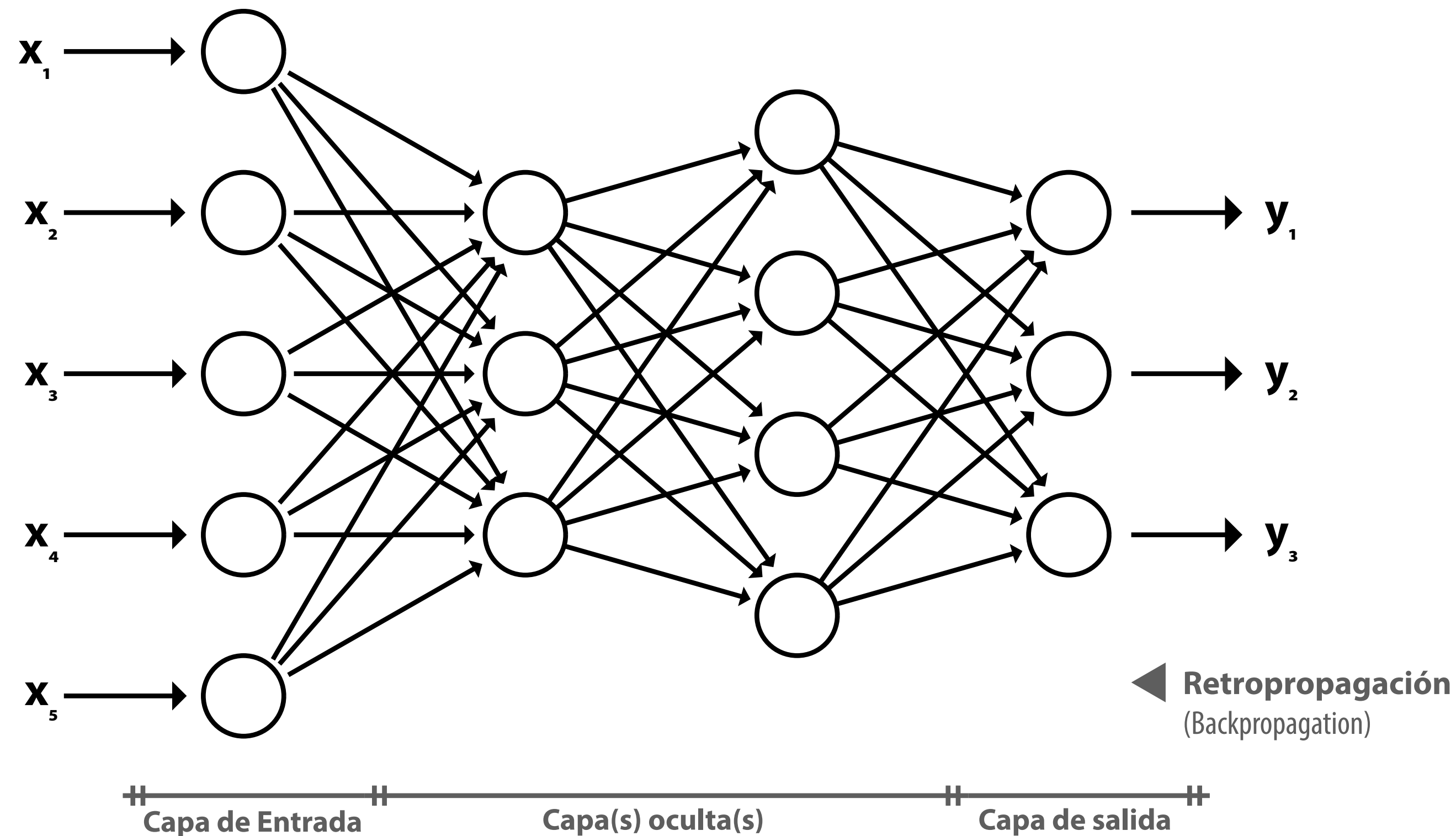


Sirve como un mecanismo de retroalimentación que le indica al modelo qué tan “erradas” son sus predicciones, siendo el objetivo final del entrenamiento **minimizar** este valor.

Capacidad/Complejidad del modelo: Redes neuronales

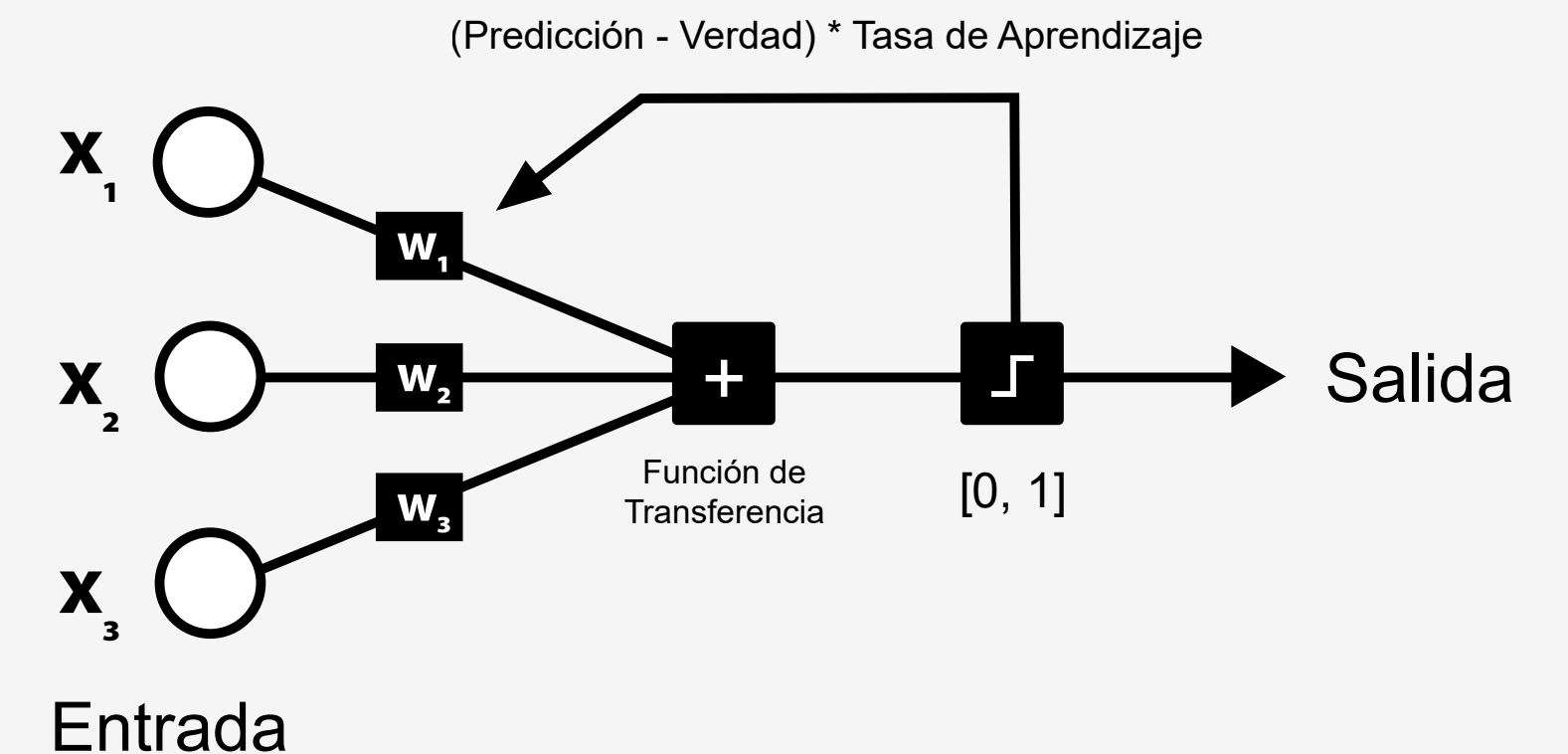
También conocidas como sistemas conexionistas, se trata de un conjunto de unidades, llamadas neuronas artificiales, conectadas entre sí para transmitirse señales.

Entrenamiento ►
(Training)



◀ **Retropropagación**
(Backpropagation)

Neuronas artificiales



Nota: Las redes neuronales "superficiales" son un término que se usa para describir NN que generalmente tienen solo una capa oculta en comparación con NN profundas que tienen varias capas ocultas, a menudo de varios tipos.

Fuentes de error: Ruido, Sesgo y Varianza

En pocas palabras, es la prueba de fuego de cualquier algoritmo es lograr la generalización. Es la diferencia entre aprender un concepto y memorizar las respuestas de un examen.

A

Ruido (Noise): Es el error inherente a los datos en sí. Ocurre porque el mundo real es impredecible, porque hay variables que no estamos midiendo o porque los datos están mal etiquetados. Es una barrera insuperable; no importa qué tan bueno sea tu modelo, nunca podrá predecir el ruido aleatorio en datos nuevos (test data).

B

Sesgo (Bias): Es el error que proviene de la rigidez del modelo. Sucede cuando el modelo que elegimos es demasiado simple para capturar la complejidad real del problema. Por ejemplo, intentar usar una simple línea recta para predecir un comportamiento que en realidad tiene forma de onda. El modelo no es lo suficientemente flexible, por lo que comete errores sistemáticos (Subajuste o Underfitting).

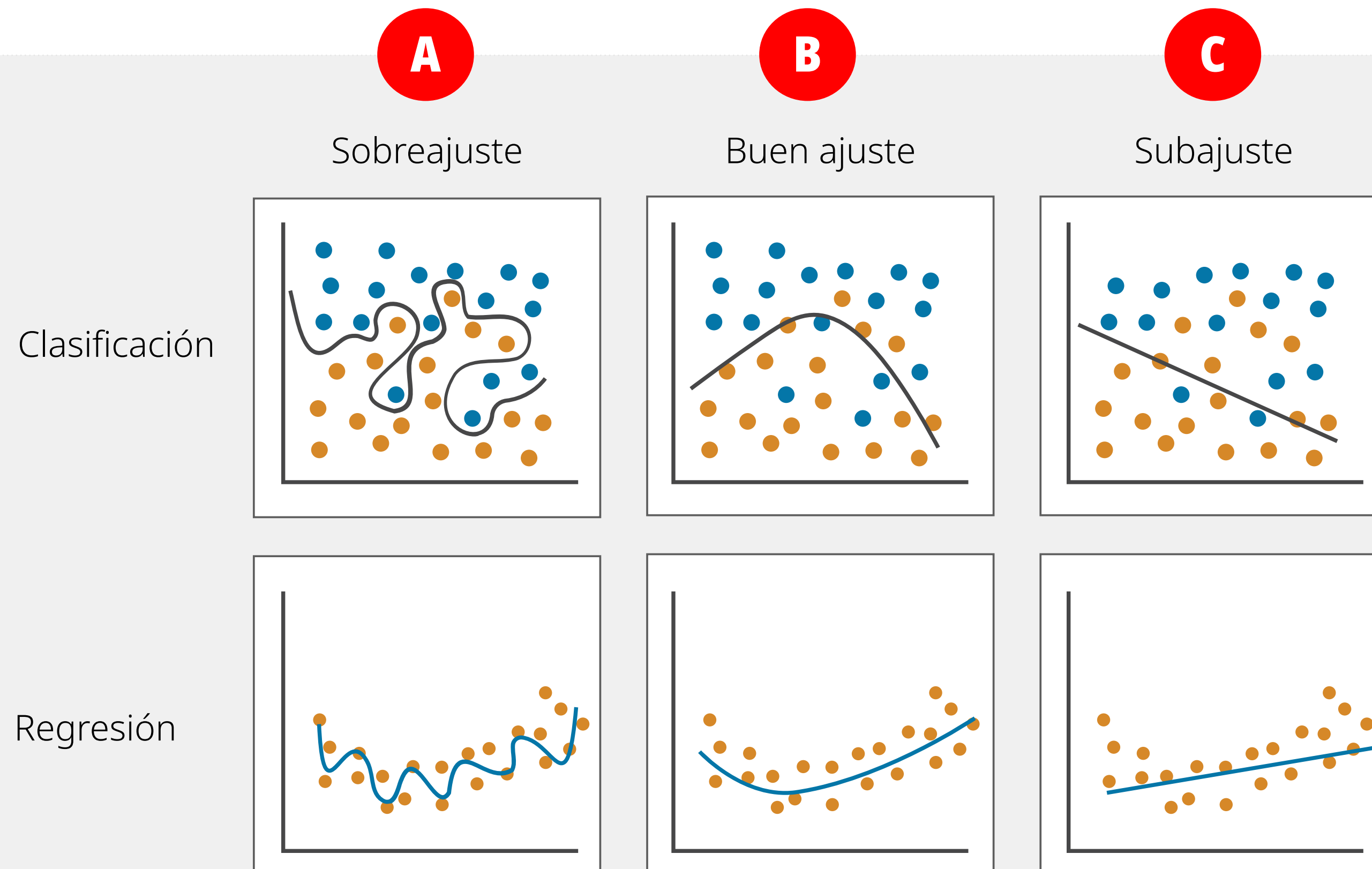
C

Varianza (Variance): Es el error que proviene de la hipersensibilidad del modelo. Sucede cuando tenemos datos de entrenamiento limitados y un modelo tan complejo que termina memorizando el ruido o las peculiaridades exactas de esos datos específicos, en lugar de aprender el patrón general. Si entrenaras el mismo modelo con un conjunto de datos ligeramente distinto, la predicción cambiaría drásticamente. (A esto también se le conoce como Sobreajuste u Overfitting).

La capacidad está directamente ligada a los parámetros. La capacidad del modelo se refiere al tamaño y la diversidad del conjunto de funciones matemáticas que el modelo es capaz de aprender y representar. En términos simples, es la "flexibilidad" o el "tamaño del cerebro" del algoritmo. Para entenderlo mejor, conectemos esto con los conceptos que acabamos de ver: 1) Capacidad Baja: Un modelo con baja capacidad solo puede entender relaciones muy simples y rígidas. Por ejemplo, una línea recta. Si le presentas un problema complejo (como el comportamiento de un mercado financiero), no tendrá la flexibilidad matemática para entenderlo. Esto resulta en Alto Sesgo (Subajuste). 2) Capacidad Alta: Un modelo con alta capacidad puede doblarse, torcerse y adaptarse a relaciones increíblemente complejas y no lineales. Sin embargo, si le das demasiada capacidad y no tienes suficientes datos, el modelo usará esa flexibilidad para memorizar el ruido y las anomalías de los datos de entrenamiento. Esto resulta en Alta Varianza (Sobreajuste). La regularización es una técnica matemática que utilizamos para evitar que un modelo sufra de sobreajuste (overfitting).

Sobreajuste, buen ajuste y subajuste

El subajuste (underfitting), el sobreajuste (overfitting) y el ajuste óptimo (best fit) describen qué tan bien un modelo de aprendizaje automático aprende los patrones de los datos de entrenamiento y los generaliza a datos nuevos y desconocidos. Lograr el ajuste óptimo requiere equilibrar la complejidad del modelo para evitar tanto el subaprendizaje rígido como la memorización caótica.

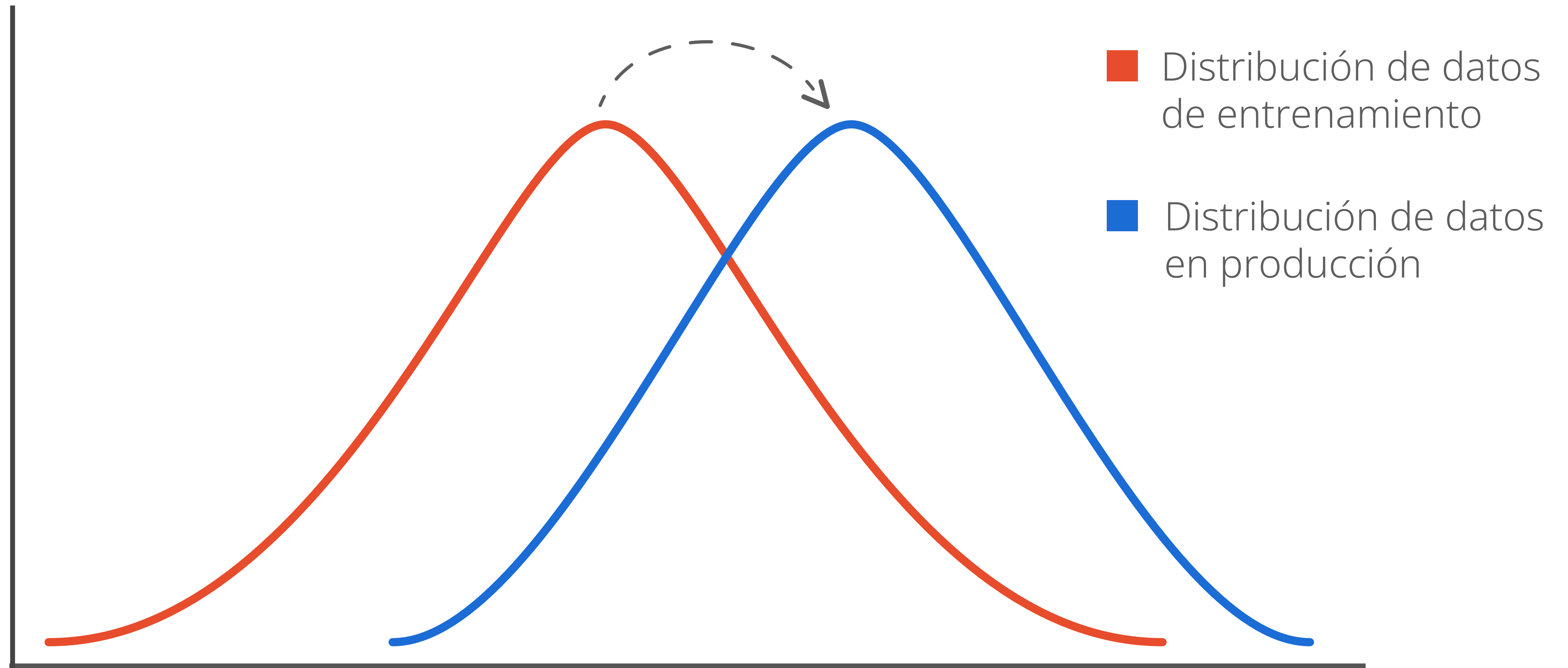


El **subajuste (underfitting)** ocurre cuando un modelo es demasiado simple y no logra capturar los patrones de los datos, generalmente por falta de capacidad algorítmica, escasez de variables o exceso de regularización. Se soluciona aumentando la complejidad del modelo, añadiendo características relevantes y reduciendo las restricciones de regularización.

El **sobreajuste (overfitting)** sucede cuando un modelo es tan complejo que memoriza los datos de entrenamiento (incluyendo el ruido) en lugar de aprender reglas generales, a menudo por usar muchos parámetros, pocos datos o entrenar en exceso. Para corregirlo, se deben usar más datos, simplificar la arquitectura del modelo, aplicar regularización o detener el entrenamiento a tiempo (early stopping).

El **ajuste óptimo (best fit)** es el equilibrio ideal en el que el modelo generaliza perfectamente ante datos nuevos, capturando la señal real y descartando el ruido. Se evidencia cuando los errores de entrenamiento y validación disminuyen a la par, estabilizándose en un punto mínimo con muy poca diferencia entre ambos.

Deriva (Data & Model Drifting)

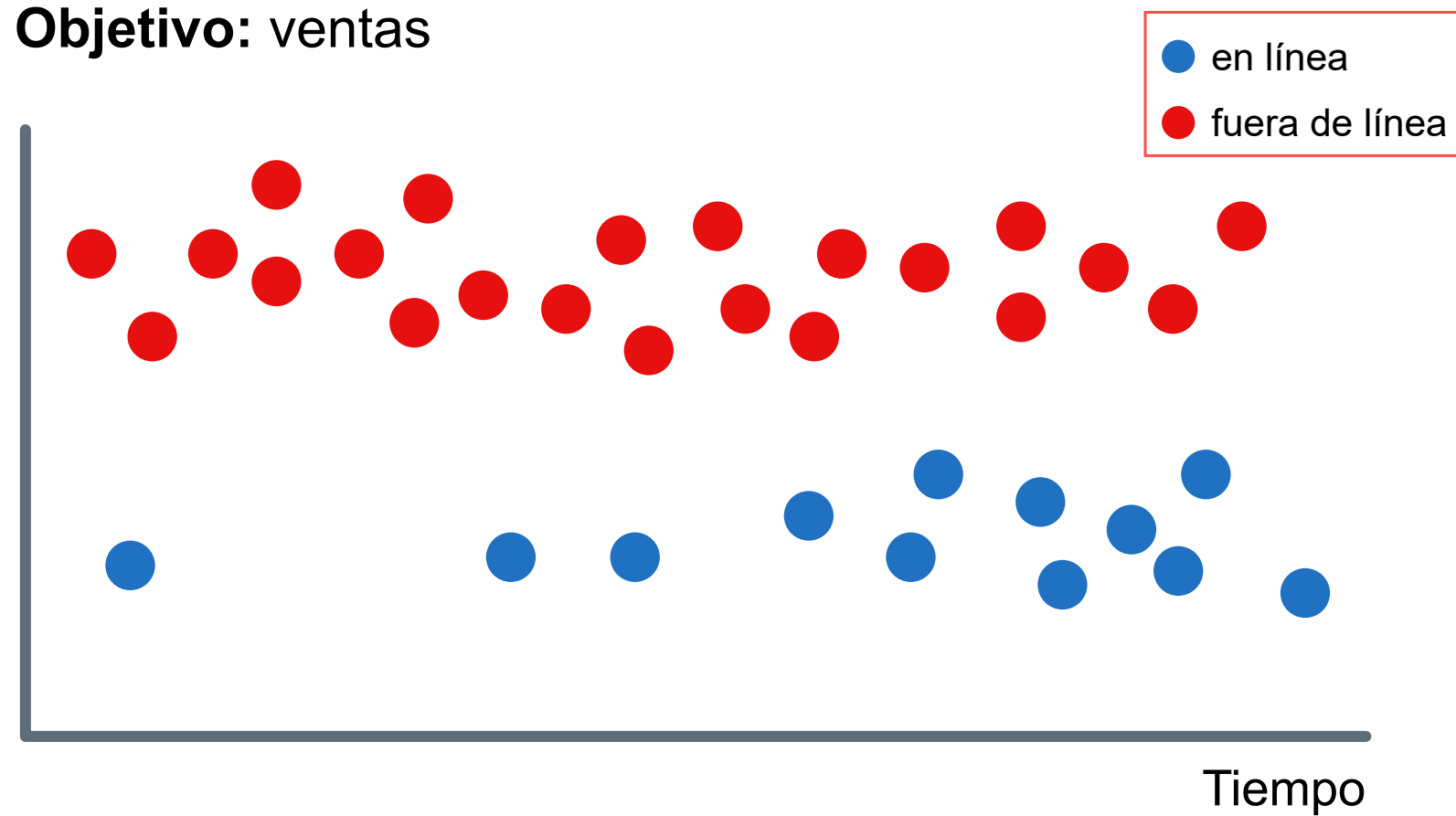


Deriva (Data & Model Drifting)

La deriva de datos (data drift) y la deriva de concepto (concept drift) suelen ocurrir simultáneamente y la primera puede ser síntoma de la segunda, aunque no siempre es así.

Deriva de datos

Objetivo: ventas

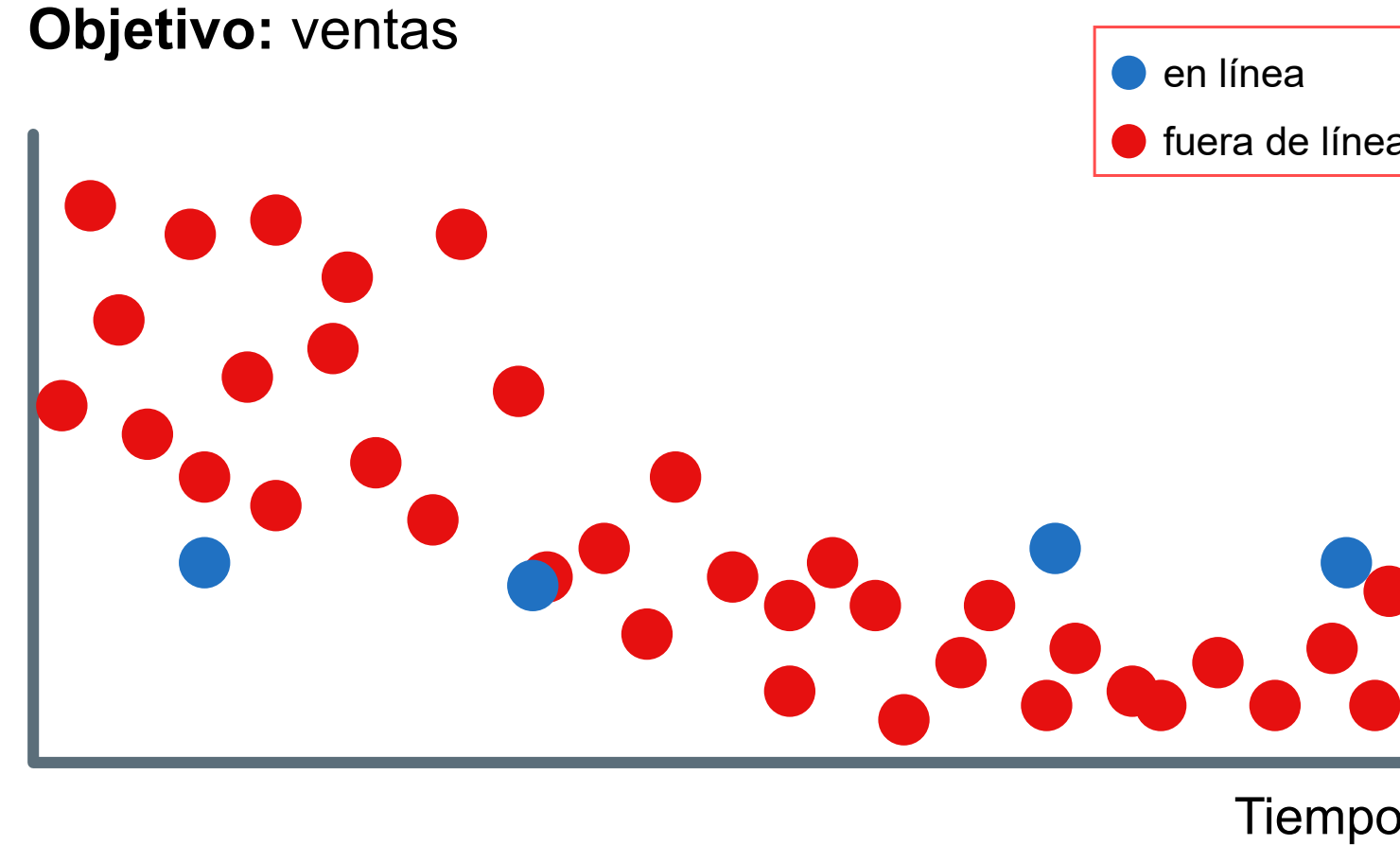


Distribución de características: canal de ventas



Deriva de concepto

Objetivo: ventas



Distribución de características: canal de ventas



La deriva de concepto implica un cambio fundamental en los patrones de comportamiento.

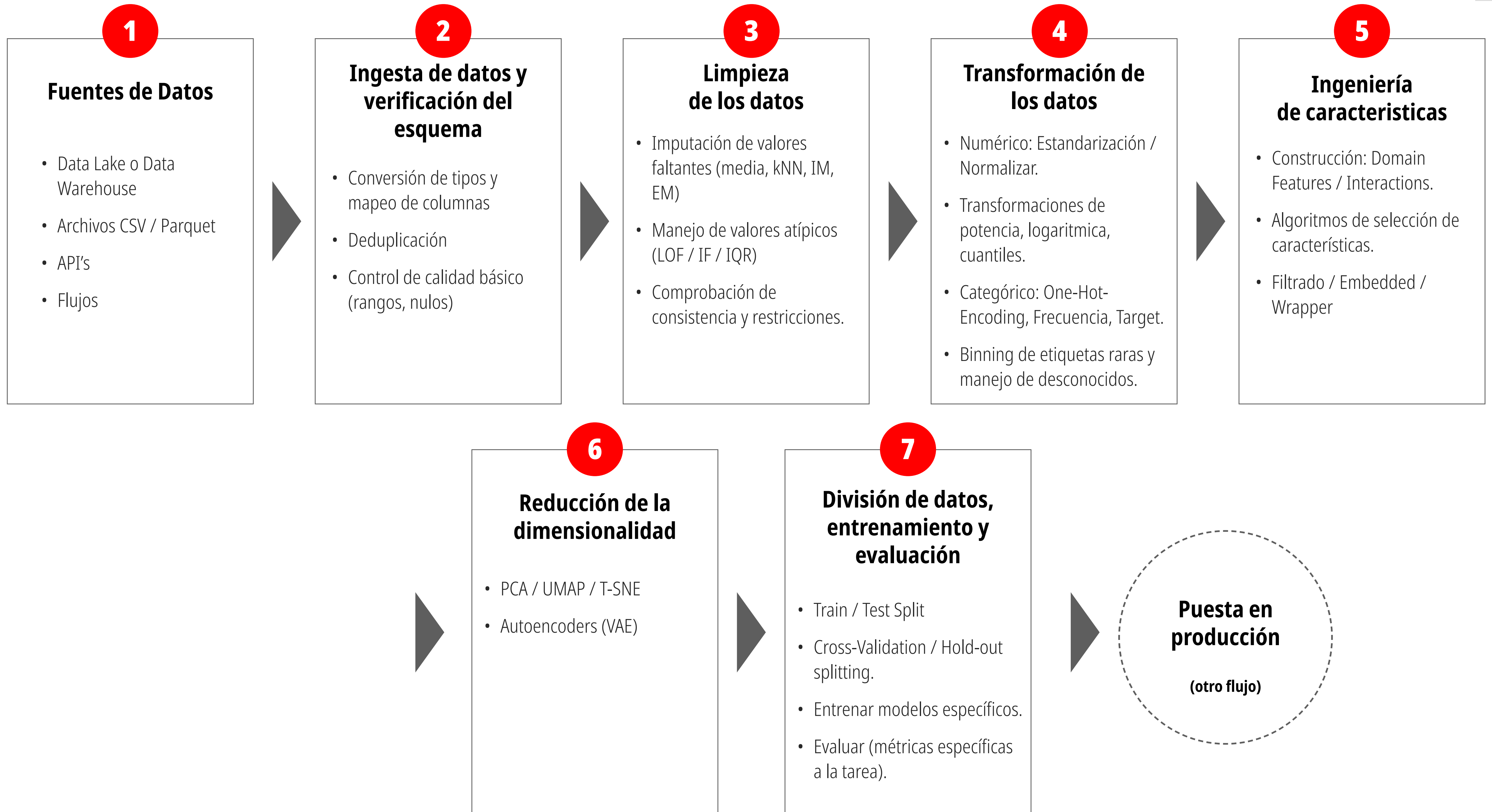
En estos casos de deriva de concepto, los modelos creados previamente quedan prácticamente obsoletos.

Deriva (Data & Model Drifting)

Cuando se habla de deriva de datos (data drift), normalmente nos referimos a las características (features) de entrada que se introducen en el modelo. En cambio, la deriva de predicción (prediction drift) es el cambio en la distribución de las salidas (outputs) del modelo.



La deriva de predicción (prediction drift) puede indicar muchos problemas, desde entradas de datos de baja calidad hasta la deriva de concepto (concept drift) en el proceso modelado. Al mismo tiempo, la deriva de predicción no siempre implica el deterioro del modelo. También puede ocurrir si el modelo se adapta bien al nuevo entorno. La diferencia: la deriva de datos se refiere a los cambios en los datos de entrada del modelo, mientras que la deriva de predicción se refiere a los cambios en las salidas del modelo.



Laboratorio 1: Flujo

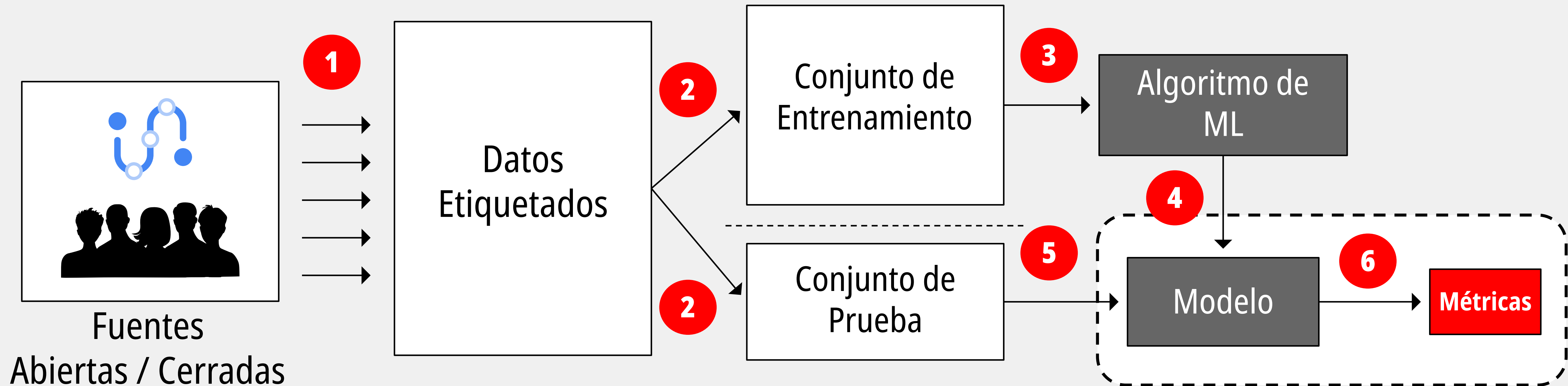
- Clasificación (Titanic)

Laboratorio 2: Generalización

- Regresión (Polinomio)
- Clasificación (Cáncer)



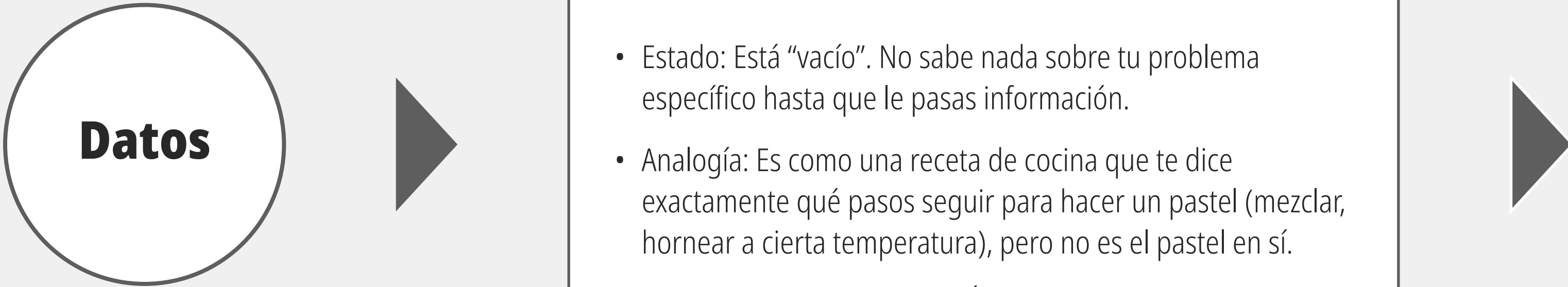
Concepto: Aprendizaje supervisado



En Machine Learning, un weak learner (aprendiz débil) es un modelo simple y ligeramente mejor que el azar (tasa de error $< 50\%$). Un strong learner (aprendiz fuerte) es un modelo complejo y altamente preciso. Los algoritmos de conjunto (ensemble methods) como Boosting combinan muchos weak learners secuencialmente para crear un modelo predictivo altamente robusto.

Desambiguación: Modelo vs Algoritmo

Es muy común confundir estos dos términos, ya que en las conversaciones del día a día suelen usarse como sinónimos. Sin embargo, en Machine Learning (Aprendizaje Automático), representan dos etapas completamente distintas del proceso.



Datos

Algoritmo

Es el conjunto de reglas matemáticas, procesos lógicos y cálculos estadísticos escritos en código. Su única función es aprender de los datos para encontrar patrones.

- Estado: Está “vacío”. No sabe nada sobre tu problema específico hasta que le pasas información.
- Analogía: Es como una receta de cocina que te dice exactamente qué pasos seguir para hacer un pastel (mezclar, hornear a cierta temperatura), pero no es el pastel en sí.
- Ejemplos: Regresión Lineal, Árboles de Decisión, Random Forest, K-Means, Redes Neuronales. Un algoritmo de Random Forest siempre funciona matemáticamente igual, sin importar si lo usas para predecir el clima o para detectar fraudes financieros.

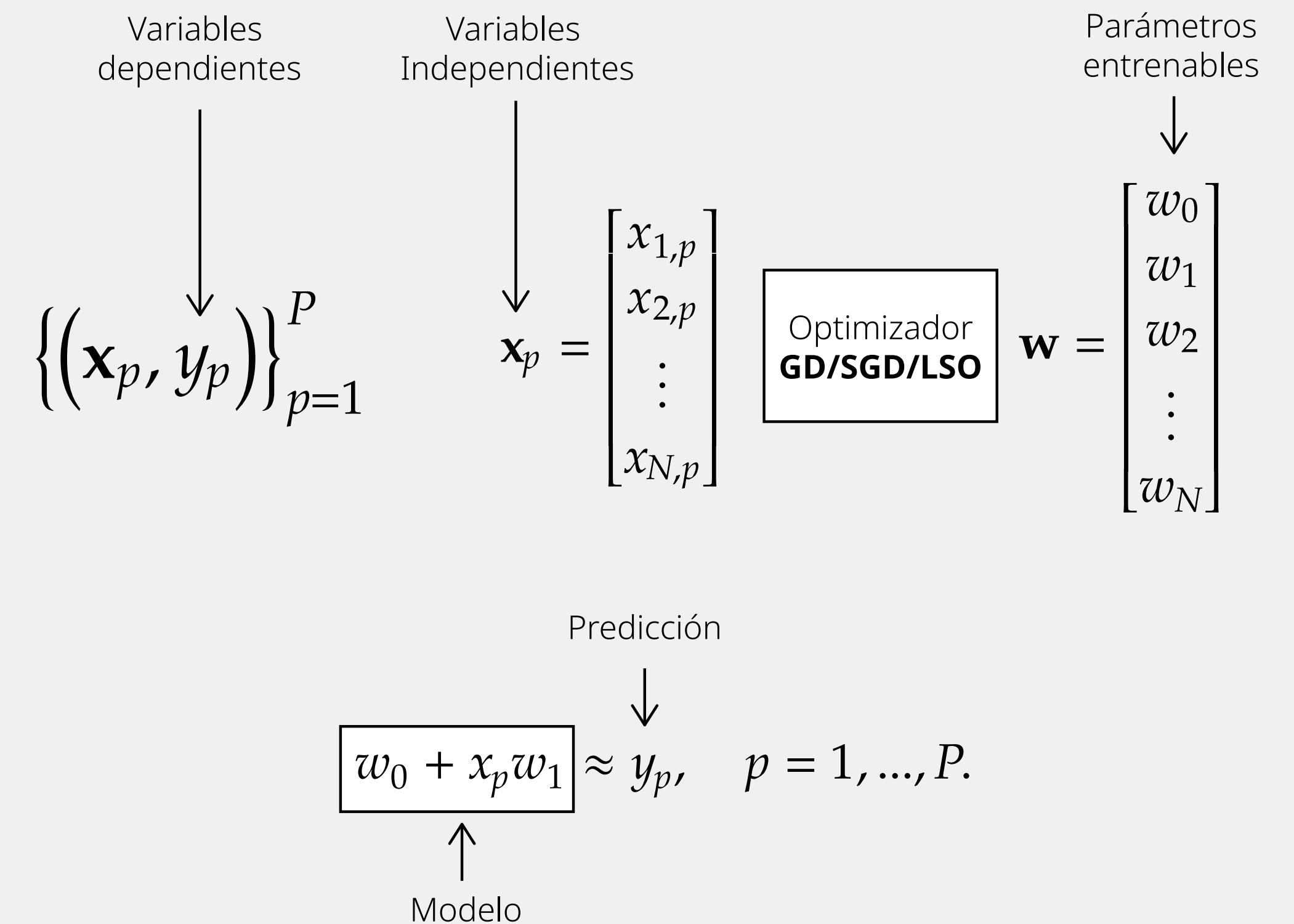
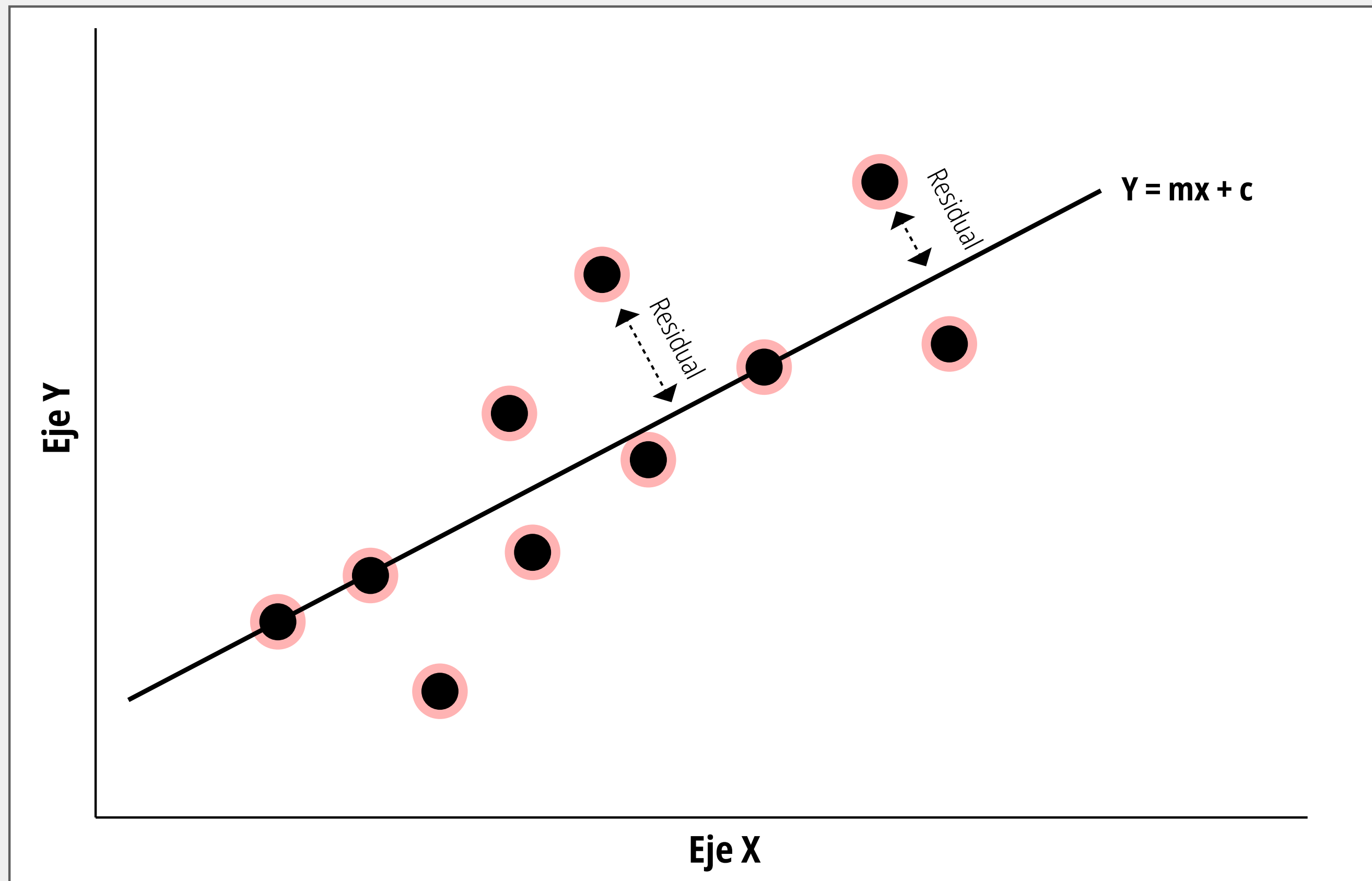
Modelo

Es el resultado (el artefacto) que obtienes después de que el algoritmo ha procesado tus datos de entrenamiento. Es la representación matemática de los patrones que el algoritmo logró extraer.

- Estado: Está “entrenado”. Contiene parámetros específicos (como pesos y sesgos en una red neuronal) que se ajustaron a tu información.
- Analogía: Es el pastel ya horneado y listo para comer. Tomaste la receta (algoritmo), le agregaste tus ingredientes específicos (datos), y el resultado es el modelo.
- Ejemplos: Un modelo de Regresión Lineal específicamente entrenado para predecir el precio de las casas en Costa Rica. Una vez que tienes el modelo, el algoritmo original ya hizo su trabajo de entrenamiento; ahora usas el modelo para hacer predicciones con datos nuevos.

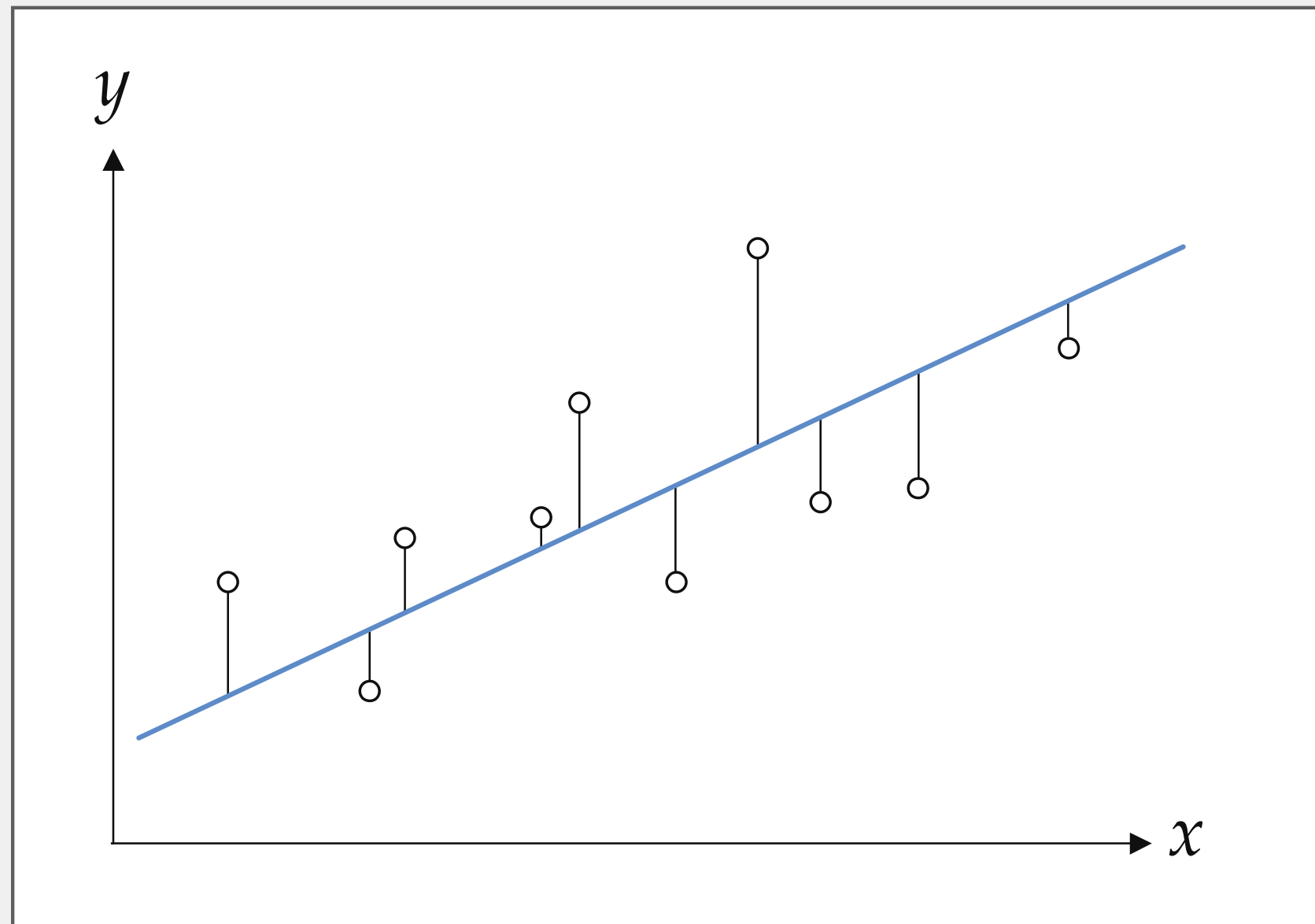
Tarea: Regresión Lineal (Forecasting)

El problema de la regresión lineal, o el ajuste de una línea representativa (o hiperplano en dimensiones superiores) a un conjunto de puntos de datos de entrada/salida.

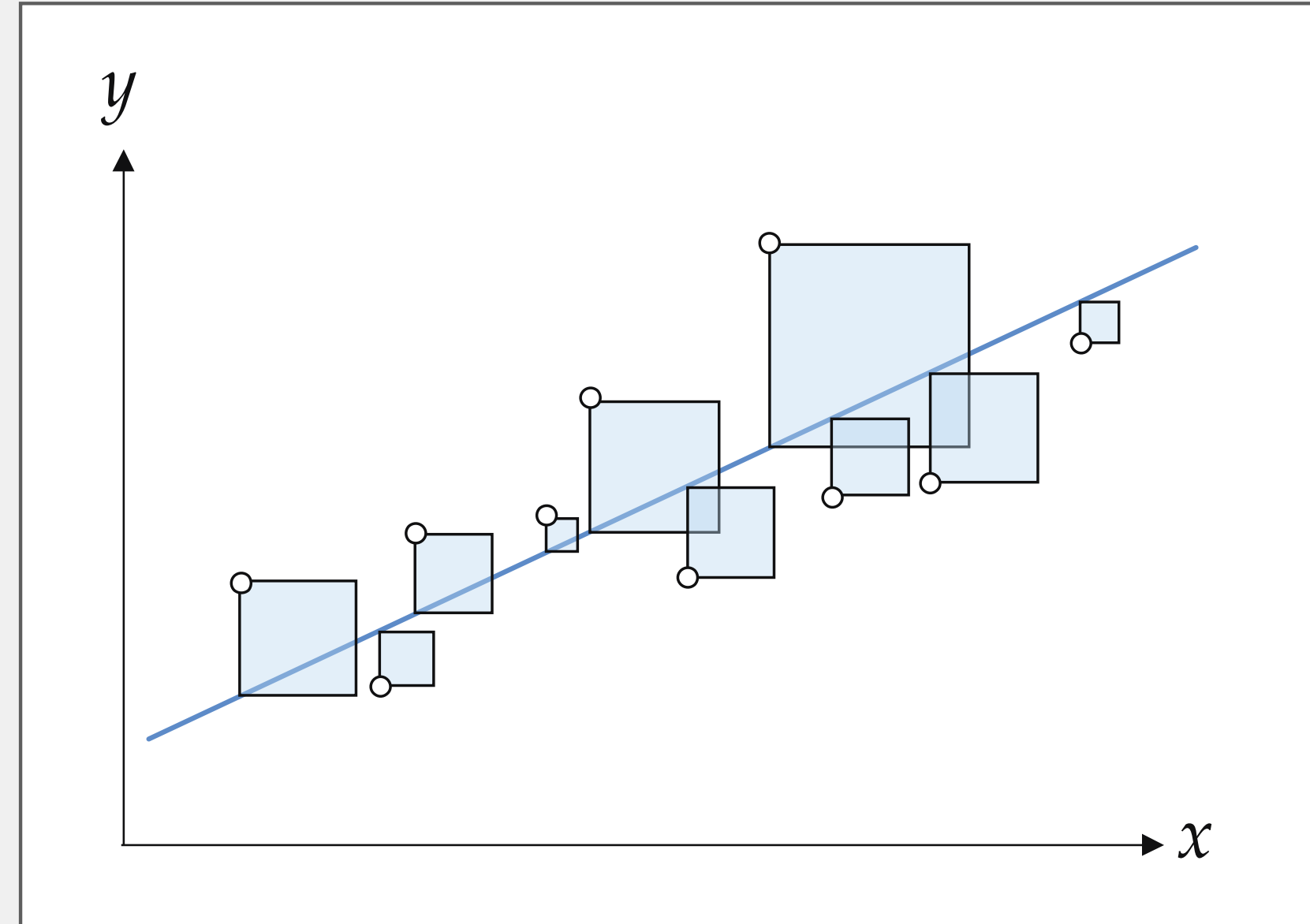


Un optimizador es un algoritmo encargado de ajustar los parámetros internos (pesos) de un modelo para minimizar el error en sus predicciones. Su objetivo principal es encontrar la configuración matemática ideal para que el modelo acierte lo más posible al comparar sus resultados con las etiquetas reales

Regresión Lineal: Mínimos Cuadrados



Un conjunto de datos bidimensional prototípico junto con una línea ajustada a los datos utilizando el marco de mínimos cuadrados (Least Squares), el cual tiene como objetivo recuperar el modelo lineal que minimiza la longitud total al cuadrado de las barras de error sólidas.



El error de mínimos cuadrados puede considerarse como el área total de los cuadrados azules, cuyos lados son las barras de error negras sólidas. La función de costo se denomina de mínimos cuadrados porque nos permite determinar un conjunto de parámetros cuya línea correspondiente minimiza la suma de estos errores al cuadrado.

Pérdida:

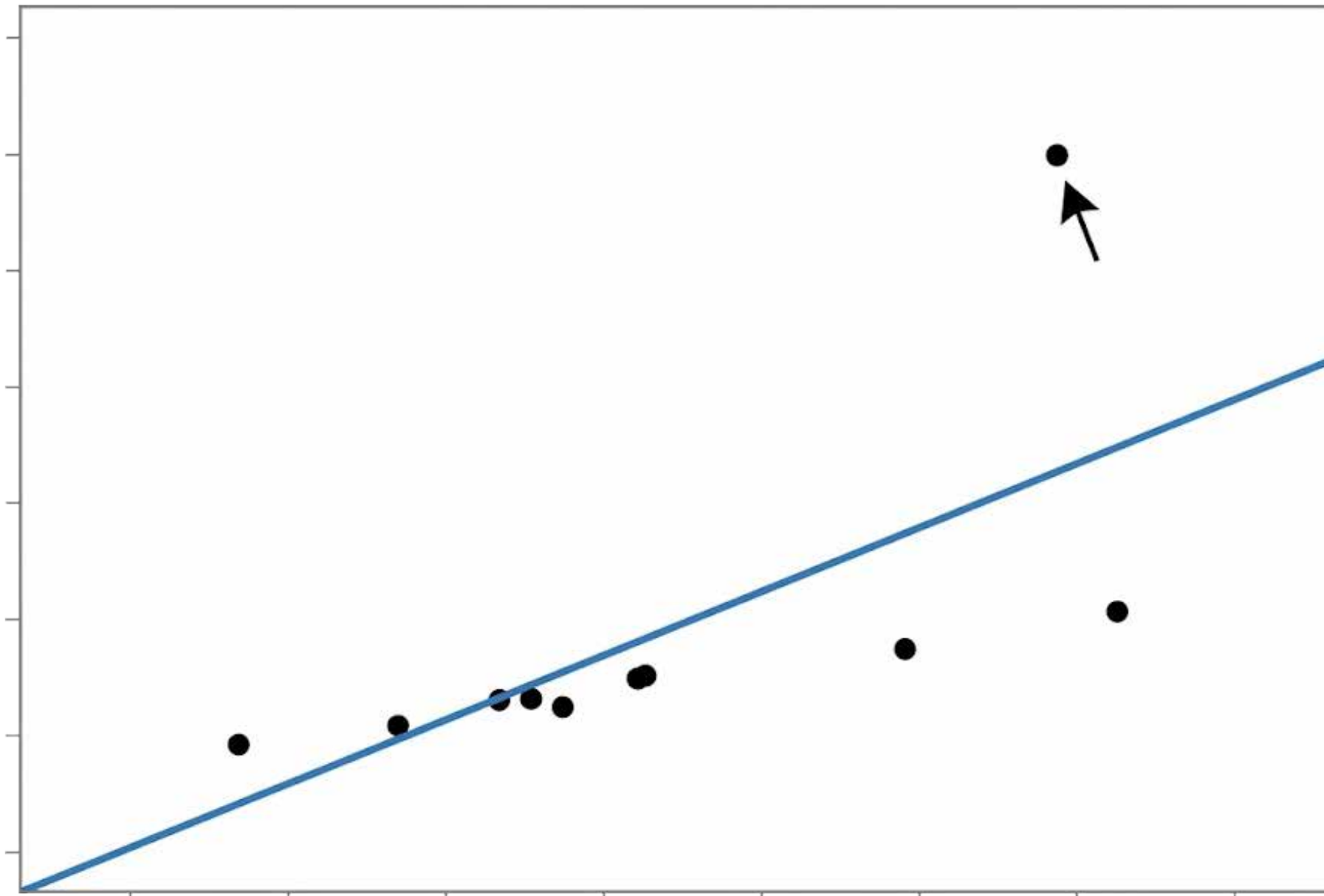
$$g(\mathbf{w}) = \frac{1}{P} \sum_{p=1}^P g_p(\mathbf{w}) = \frac{1}{P} \sum_{p=1}^P (\mathbf{x}_p^T \mathbf{w} - y_p)^2$$

$$\underset{\mathbf{w}}{\text{minimize}} \quad \frac{1}{P} \sum_{p=1}^P (\mathbf{x}_p^T \mathbf{w} - y_p)^2$$

- Convexidad: Si una función es convexa, tiene la forma de un tazón perfecto. Esto significa que solo tiene un punto más bajo (el mínimo global). No hay “valles falsos” (mínimos locales) donde un algoritmo pueda quedarse atascado.
- Infinitamente diferenciable: Significa que la curva es completamente suave. No tiene picos repentinos ni cortes abruptos.
- El resultado: Gracias a estas dos propiedades, matemáticamente puedes soltar cualquier algoritmo de optimización en esta función y, eventualmente, se deslizará hasta el fondo exacto para encontrar los pesos óptimos del modelo.

Regresión Lineal: Error absoluto

Una desventaja de usar el error cuadrático en el costo de Mínimos Cuadrados (como medida que luego minimizamos para obtener los parámetros óptimos de regresión lineal) es que elevar el error al cuadrado aumenta la importancia de los errores más grandes.



Al elevar los errores al cuadrado, esta función castiga de forma desproporcionada los errores grandes. Esto provoca que el modelo se desvíe intentando adaptarse a los valores atípicos (outliers), lo que genera un sobreajuste.

$$g_p(\mathbf{w}) = (\mathbf{x}_p^T \mathbf{w} - y_p)^2 \quad p = 1, \dots, P$$

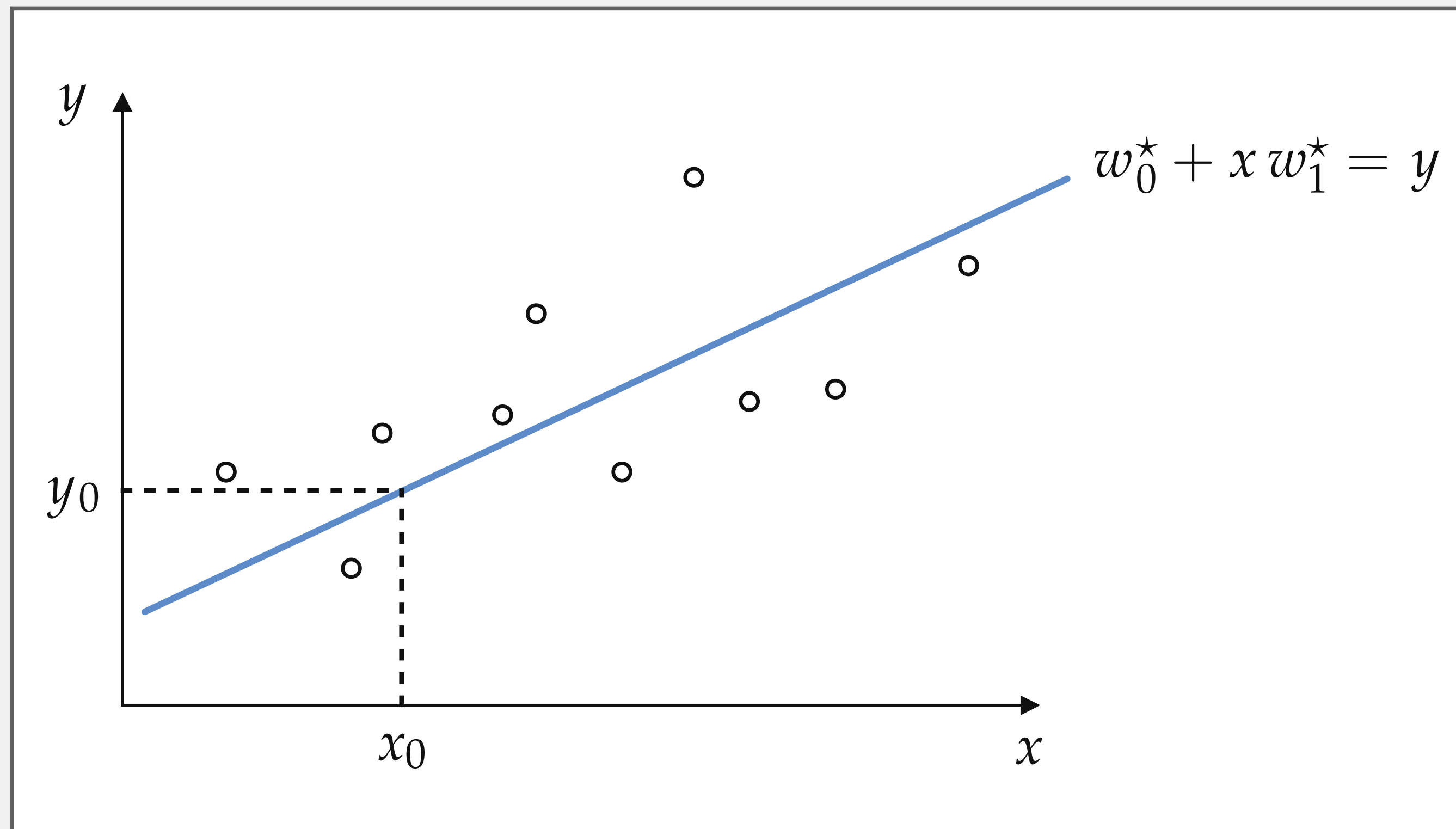
$$g_p(\mathbf{w}) = |\mathbf{x}_p^T \mathbf{w} - y_p| \quad p = 1, \dots, P.$$

$$g(\mathbf{w}) = \frac{1}{P} \sum_{p=1}^P g_p(\mathbf{w}) = \frac{1}{P} \sum_{p=1}^P |\mathbf{x}_p^T \mathbf{w} - y_p|.$$

Aunque la función de error absoluto es convexa, su segunda derivada es cero en casi todas partes. Esto nos impide utilizar métodos de optimización rápidos y avanzados (los de segundo orden), obligándonos a depender exclusivamente de algoritmos de orden cero o de primer orden.

Regresión Lineal: Evaluación

Una vez que hemos minimizado exitosamente una función de costo de regresión lineal, resulta sencillo determinar la calidad de nuestro modelo de regresión: simplemente evaluamos una función de costo utilizando nuestros pesos óptimos.



Para ello, introducimos los parámetros aprendidos del modelo junto con los datos en el costo de Mínimos Cuadrados, obteniendo el llamado Error Cuadrático Medio (o MSE, por sus siglas en inglés).

$$\text{MSE} = \frac{1}{P} \sum_{p=1}^P \left(\text{model}(\mathbf{x}_p, \mathbf{w}^*) - y_p \right)^2.$$

Para reducir el efecto de los valores atípicos (outliers) y otros valores grandes, a menudo se extrae la raíz cuadrada de este valor para usarla como métrica de calidad de regresión. A esto se le conoce como la Raíz del Error Cuadrático Medio (RMSE).

$$\text{MAD} = \frac{1}{P} \sum_{p=1}^P \left| \text{model}(\mathbf{x}_p, \mathbf{w}^*) - y_p \right|.$$

De igual manera, podemos emplear el costo de Mínimas Desviaciones Absolutas para determinar la calidad de nuestro modelo entrenado. Al introducir los parámetros aprendidos del modelo junto con los datos en este costo, se calculan las Desviaciones Absolutas Medias (o MAD, por sus siglas en inglés), que es precisamente lo que calcula esta función de costo.

Interludio: Meta-Learning

Después de desarrollar un único modelo, se podría pensar en cómo seguir mejorando su rendimiento. Un método que constantemente ha brindado una mejora en el rendimiento es utilizar un ensamble (o conjunto) de múltiples modelos en lugar de solo un modelo individual para realizar predicciones. Cada modelo en el ensamble se denomina aprendiz base (base learner).

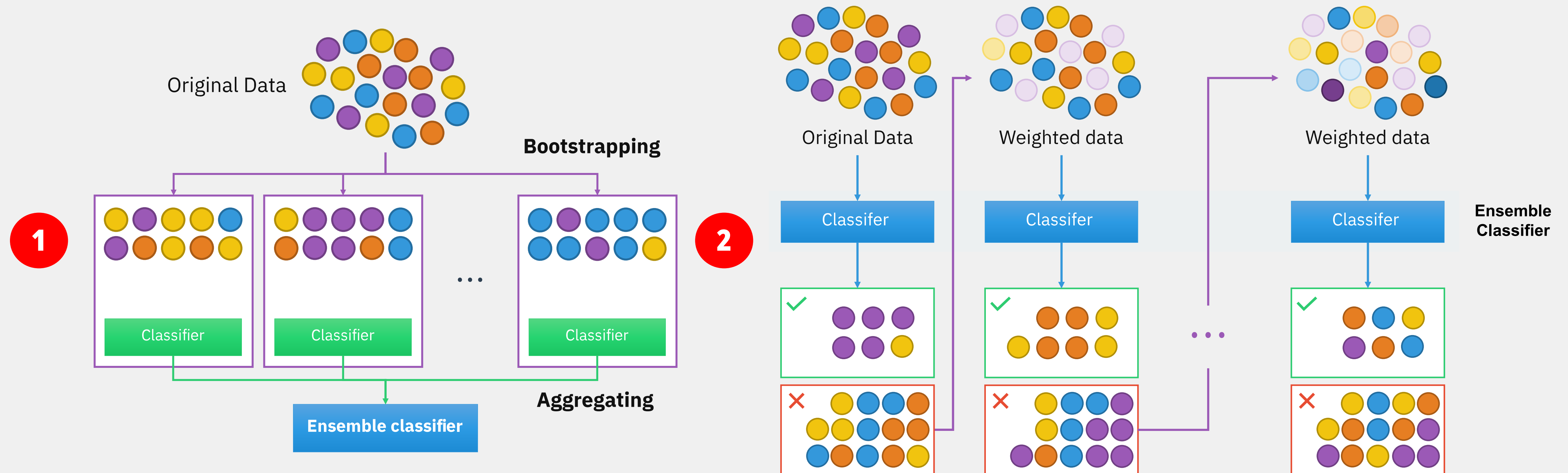
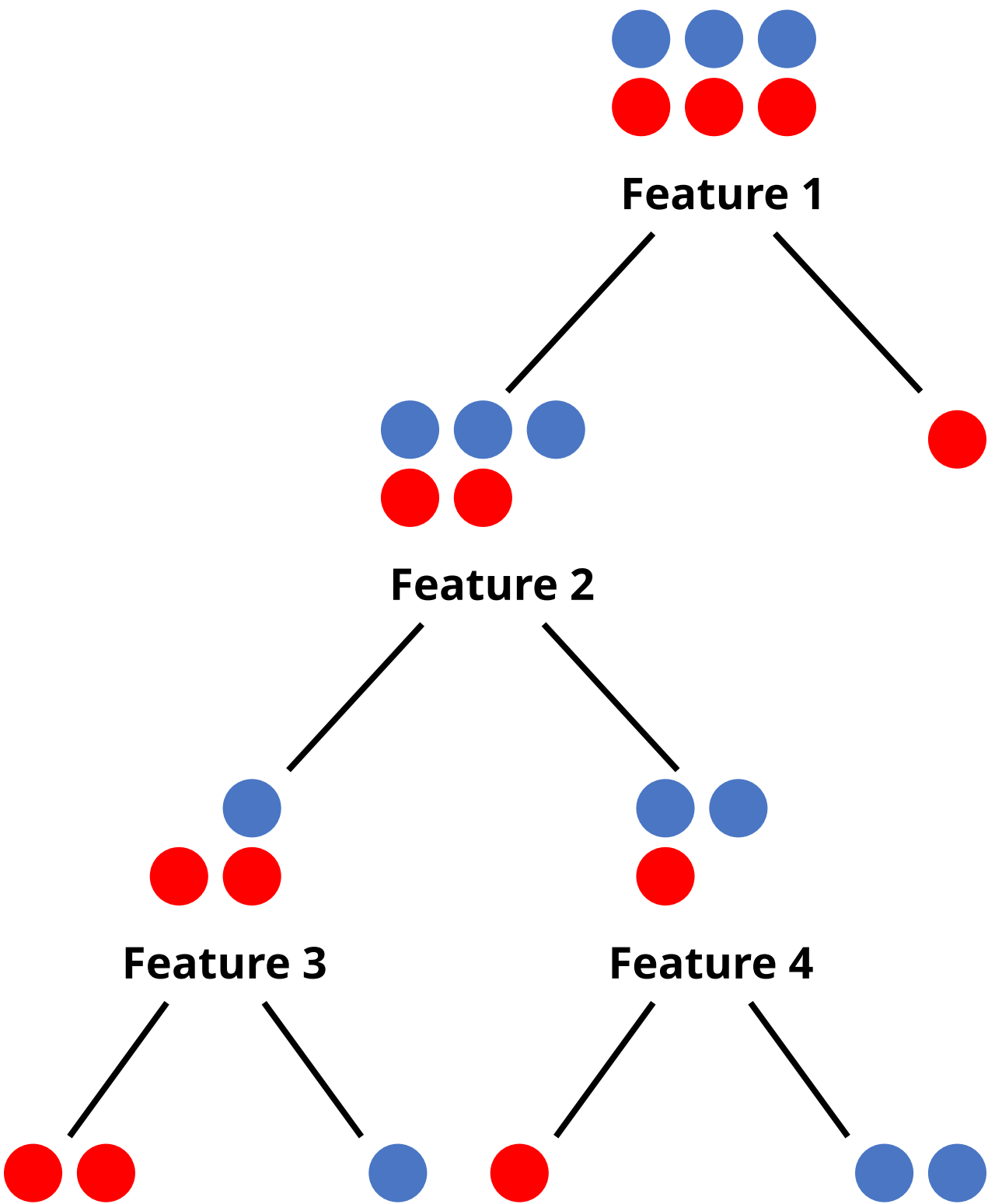


Figura: Bagging (1) y Boosting (2).

Los métodos de ensamble son menos preferidos en producción porque los ensambles son más complejos de desplegar y más difíciles de mantener. Sin embargo, siguen siendo comunes para tareas en las que una pequeña mejora en el rendimiento puede generar una enorme ganancia financiera, como predecir la tasa de clics (click-through rate) en los anuncios. Existen tres formas de crear un ensamble: bagging, boosting y stacking. Además de ayudar a mejorar el rendimiento, según varios artículos de revisión, los métodos de ensamble como boosting y bagging, junto con el remuestreo (resampling), han demostrado ser útiles con conjuntos de datos desbalanceados. En bagging se toman muestras con reemplazo para crear diferentes conjuntos de datos, llamados bootstraps. En boosting las muestras se vuelven a ponderar en función de qué tan bien las clasifica el primer clasificador; por ejemplo, a las muestras mal clasificadas se les asigna un peso mayor.

Regresión y Clasificación: CART

Las clases `DecisionTreeRegressor` y `DecisionTreeClassifier` implementan una versión optimizada del algoritmo CART (Classification and Regression Trees, o Árboles de Clasificación y Regresión).



A diferencia de otros algoritmos clásicos de árboles como ID3 o C4.5 (que pueden tener múltiples ramas por nodo), la implementación CART de scikit-learn construye estrictamente árboles binarios. Esto significa que cada nodo interno siempre evalúa una condición de verdadero/falso y se divide exactamente en dos ramas secundarias (por ejemplo, “¿La edad es mayor a 30?”).

Parámetro	Algoritmo	Descripción y Uso Estratégico
criterion	Clasificador	Mide la calidad del corte. Opciones: ‘gini’ (por defecto, más rápido), ‘entropy’ (ganancia de información) o ‘log_loss’.
criterion	Regresor	Opciones: ‘squared_error’ (MSE, por defecto), ‘absolute_error’ (MAE, más robusto a valores atípicos), ‘friedman_mse’ o ‘poisson’.
max_depth	Ambos	Profundidad máxima del árbol. Crítico para el control de memoria y sobreajuste. Si es None, los nodos se expanden hasta que las hojas sean puras, lo cual es computacionalmente costoso y garantiza sobreajuste en datasets complejos.
min_samples_split	Ambos	Mínimo de muestras necesarias para dividir un nodo interno (por defecto 2). Aumentar este valor (ej. 10 o 50) suaviza el modelo y evita crear reglas matemáticas para casos demasiado específicos.
min_samples_leaf	Ambos	Mínimo de muestras que deben quedar en un nodo hoja. Si una división deja una hoja con menos elementos, no se realiza. Excelente para filtrar ruido.
max_features	Ambos	Número de características a considerar al buscar el mejor corte. Opciones: ‘sqrt’, ‘log2’, o un número entero. Reduce drásticamente el tiempo de cómputo en conjuntos de datos anchos y añade aleatoriedad estructural.
class_weight	Clasificador	Fundamental para cuando las clases están fuertemente desbalanceadas . Pasarle ‘balanced’ ajusta automáticamente los pesos de las clases inversamente proporcionales a su frecuencia en los datos de entrada, forzando al modelo a prestar atención a la clase minoritaria.

Regresión y Clasificación: RandomForest (Classifier / Regressor)

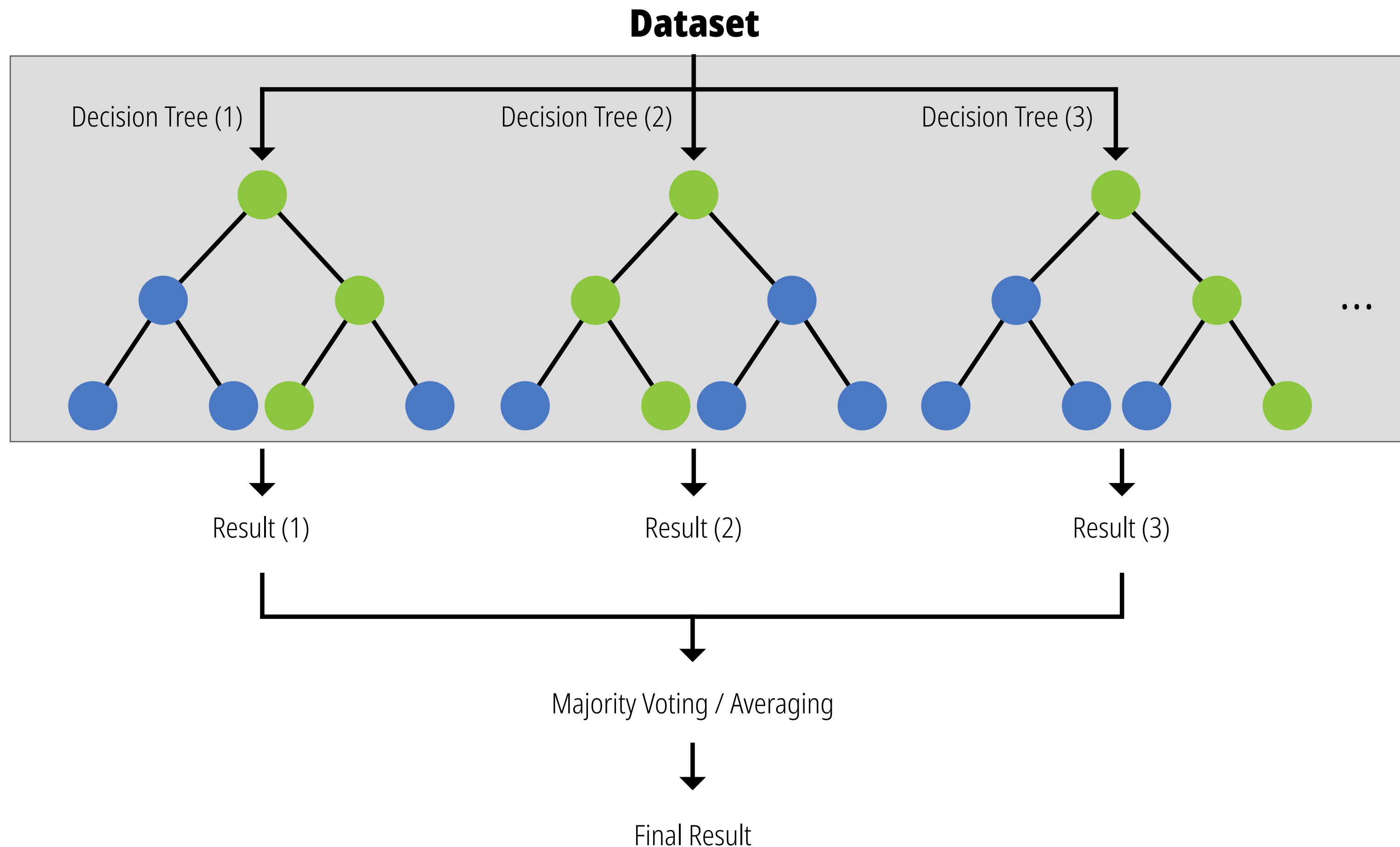
Parámetros del Bosque (ensamble)

Controlan cómo se construye el grupo de árboles en su totalidad.

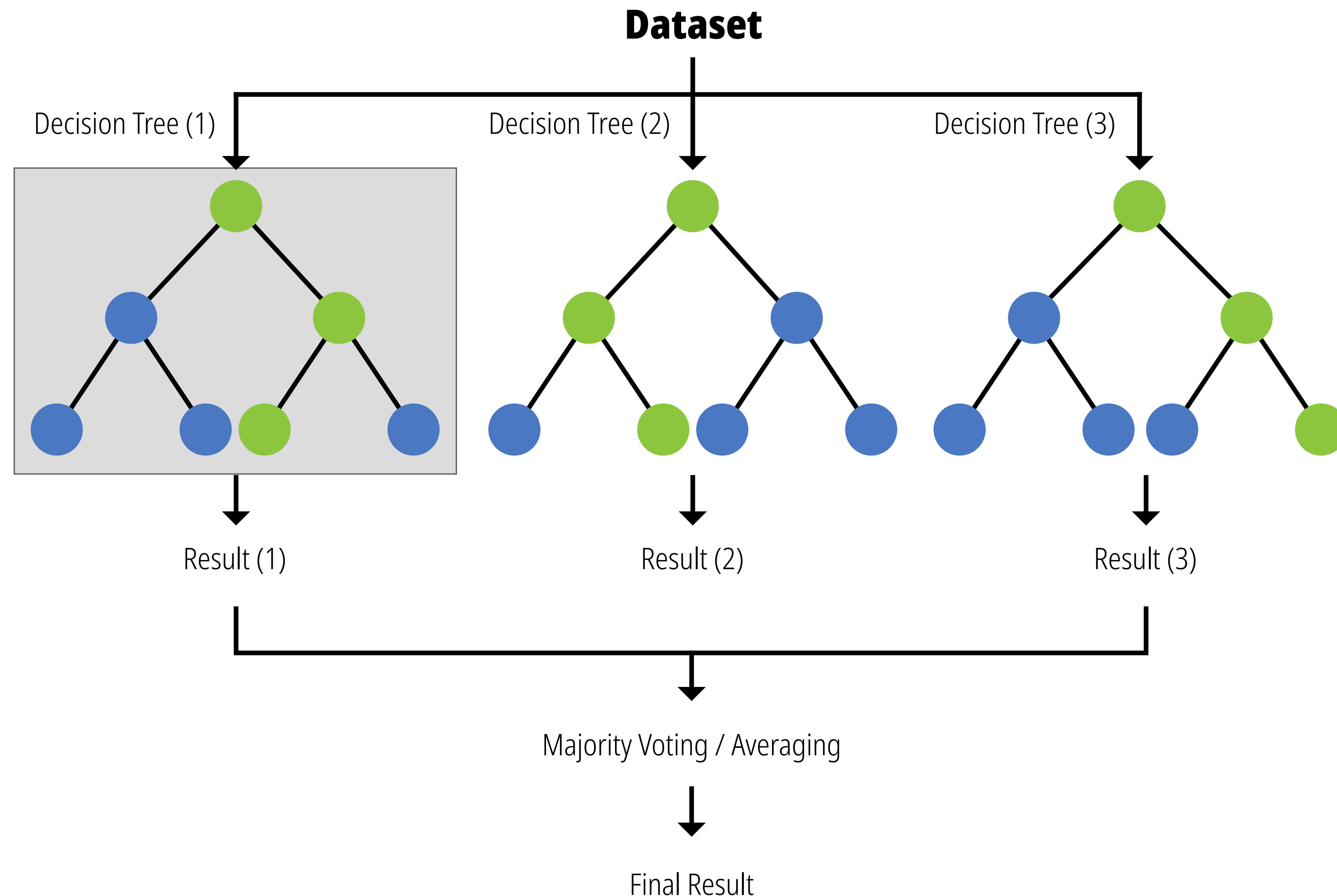
n_estimators (Por defecto: 100): Es la cantidad de árboles en el bosque. En general, más árboles significan un modelo más robusto y estable frente a variaciones en los datos. La ventaja del Random Forest es que aumentar este valor rara vez causa sobreajuste (overfitting), aunque sí incrementa el tiempo de entrenamiento y el uso de memoria.

bootstrap (Por defecto: True): Determina si los árboles se entrenan usando submuestras aleatorias de los datos originales con reemplazo. Si lo cambias a False, todo el conjunto de datos original se utilizará para construir cada árbol.

oob_score (Por defecto: False): Si usas bootstrap, siempre quedarán datos sin utilizar durante el entrenamiento de un árbol específico (conocidos como datos Out-of-Bag). Si activas este parámetro, el modelo usará esos datos sobrantes para autoevaluar su precisión sin necesidad de que dividas un conjunto de validación por separado.



Regresión y Clasificación: RandomForest (Classifier / Regressor)



Parámetros de los Árboles (Control de crecimiento)

Estos parámetros le dicen a cada árbol individual qué tanto puede crecer. Son tu principal herramienta para controlar el sobreajuste.

max_depth (Por defecto: None): Controla la profundidad máxima de cada árbol. Si lo dejas en None, los nodos se expandirán hasta que todas las hojas contengan una sola clase (lo que suele llevar a un sobreajuste masivo). Limitar este valor fuerza al modelo a encontrar patrones más generales.

...

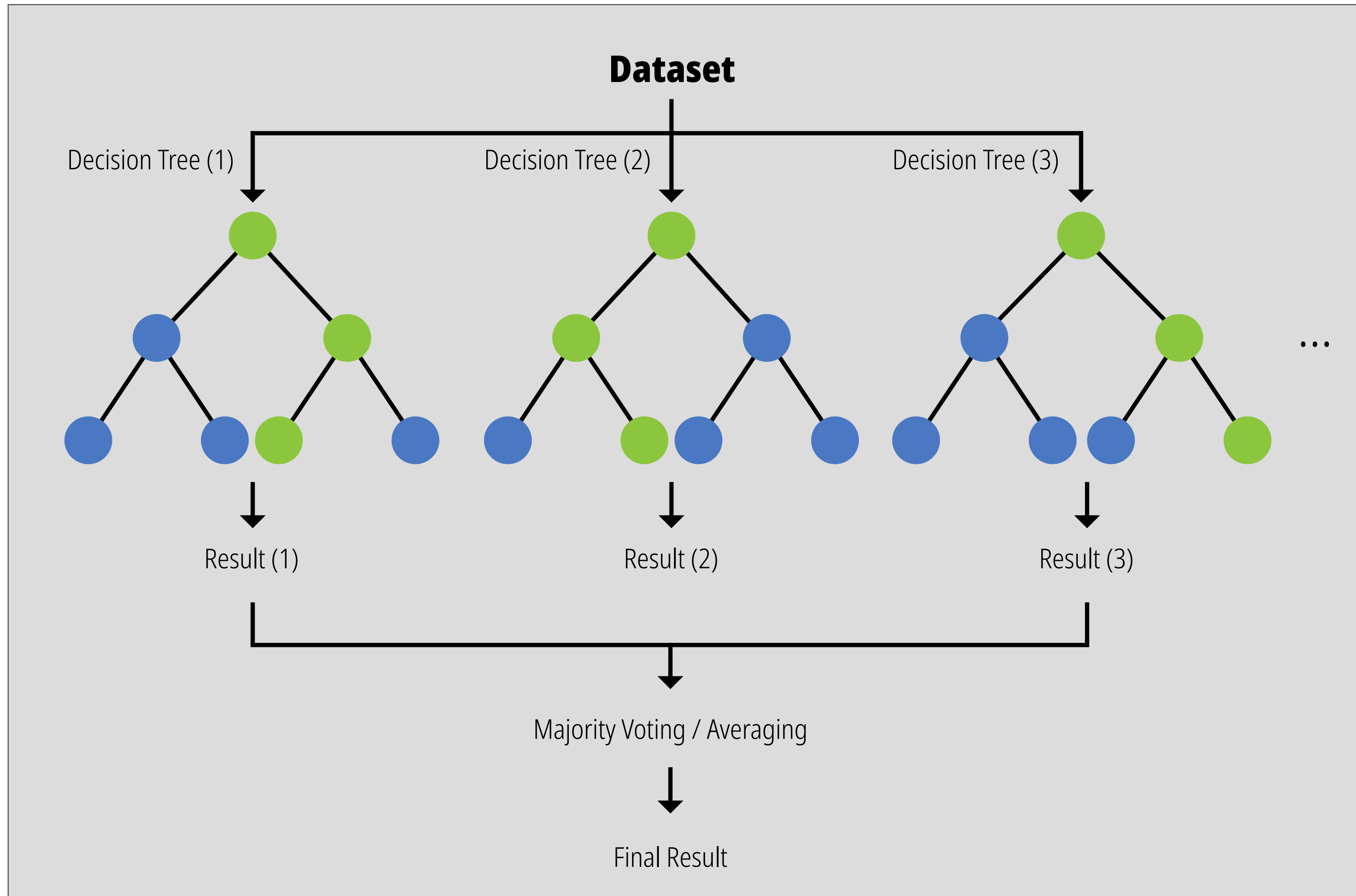
min_samples_split (Por defecto: 2): El número mínimo de muestras que un nodo interno debe tener para que se le permita dividirse en dos nuevas ramas.

min_samples_leaf (Por defecto: 1): El número mínimo de datos que debe quedar al final de una rama (en un nodo hoja). Aumentar este valor suaviza las predicciones del modelo y evita que el árbol cree reglas ultra-específicas para un solo dato anómalo.

max_features (Por defecto: 'sqrt'): El número máximo de características (columnas) que cada árbol puede evaluar al buscar la mejor forma de dividir los datos. Limitar esto es clave: garantiza que los árboles del bosque sean diferentes entre sí (decorrelacionados), lo que permite que el ensamble funcione correctamente.

Nota de implementación: Cuando configuras un parámetro del árbol (como `max_depth=5`) directamente dentro de la instancia del `RandomForestRegressor`, el bosque se encarga de inyectar esa regla internamente a los 100 árboles (o los que indiques en `n_estimators`) que genere durante el tiempo de ejecución.

Regresión y Clasificación: RandomForest (Classifier / Regressor)



Parámetros Computacionales y Prácticos

No afectan las matemáticas del modelo, pero sí la forma en que lo programas y ejecutas.

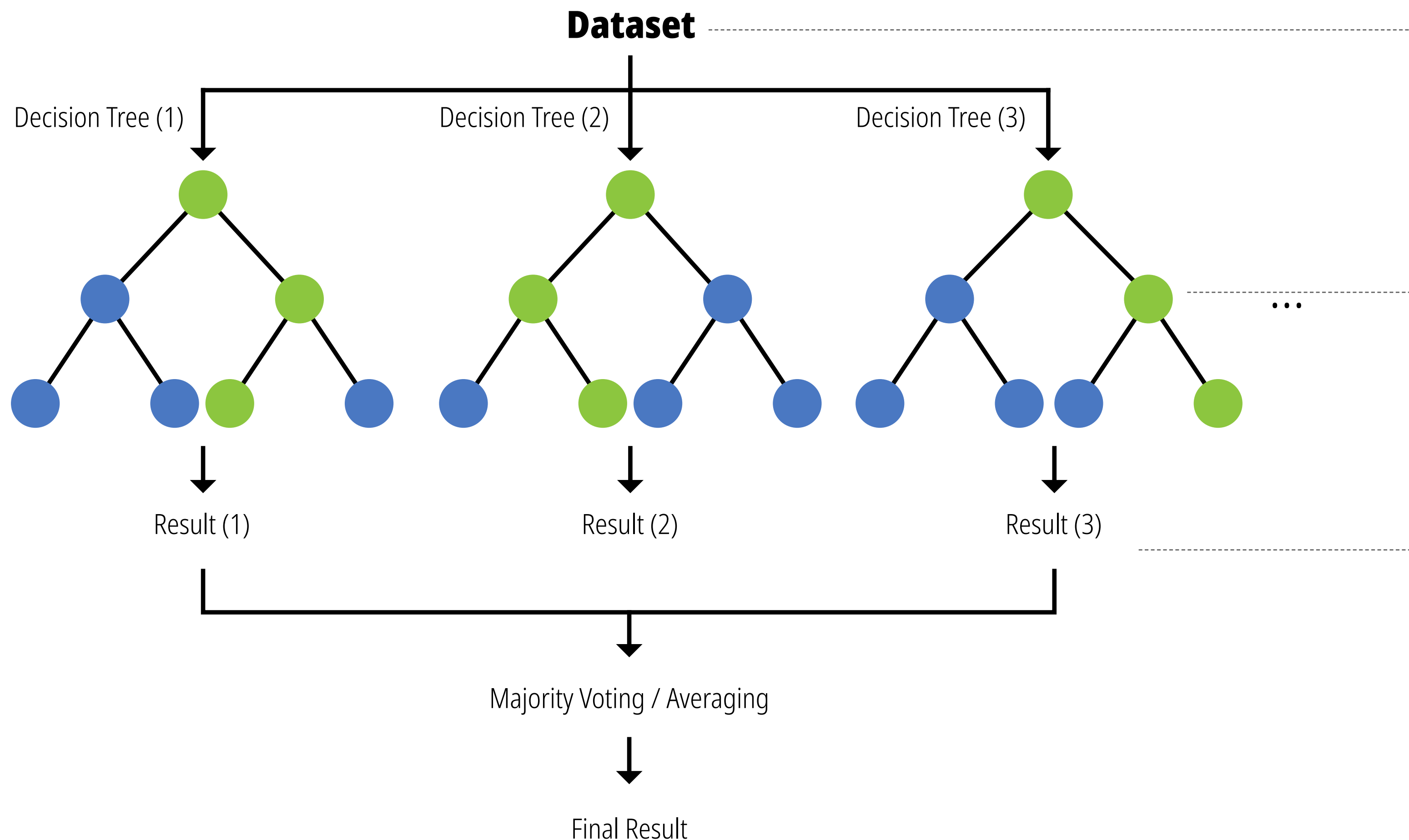
n_jobs (Por defecto: None): El número de núcleos de tu procesador que se usarán para entrenar el modelo en paralelo. Si le asignas -1, scikit-learn usará todos los núcleos disponibles en tu computadora, lo que acelera el entrenamiento drásticamente.

random_state (Por defecto: None): La semilla del generador de números aleatorios. Fijar un número (por ejemplo, 42 o 0) garantiza que si corres el código nuevamente, obtendrás exactamente los mismos resultados.

class_weight (Por defecto: None): Muy útil si tienes datos desbalanceados (por ejemplo, 99% de transacciones legítimas y 1% de fraudes). Puedes configurarlo como 'balanced' para que el algoritmo ajuste automáticamente los pesos y le dé más importancia a la clase minoritaria durante el entrenamiento.

Regresión y Clasificación: RandomForest (Classifier / Regressor)

Un árbol de Random Forest se ramifica aplicando divisiones binarias recursivas (haciendo preguntas de tipo "sí" o "no" sobre las variables). Para garantizar que cada árbol sea único, el algoritmo utiliza dos técnicas: selección aleatoria de datos y selección aleatoria de variables.



Muestreo aleatorio (Bagging): Para crear cada árbol, el algoritmo selecciona una muestra de los datos originales con reemplazo (los datos pueden repetirse). Esto significa que cada árbol se entrena con un subconjunto de datos ligeramente distinto.

1

Selección aleatoria de características: En cada nodo donde el árbol necesita ramificarse, el algoritmo no evalúa todas las variables disponibles para decidir el mejor corte. Elige aleatoriamente un grupo más pequeño (por ejemplo, la raíz cuadrada del total de variables) y solo busca la mejor división entre estas.

2

Criterio de división: Para saber exactamente dónde partir la rama, el algoritmo busca la pregunta que logre la mayor ganancia de información. Utiliza fórmulas matemáticas para medir qué tan puros quedan los grupos tras la división.

3

Regresión y Clasificación: RandomForest (Classifier / Regressor)

Las métricas matemáticas (Gini, Entropía o Varianza) no se aplican al bosque entero a la vez, sino en cada bifurcación individual de cada árbol mientras se están construyendo.

$$Gini = 1 - \sum_{i=1}^C p_i^2$$

$$Entropía = - \sum_{i=1}^C p_i \log_2(p_i)$$

$$Varianza = \frac{1}{N} \sum_{i=1}^N (y_i - \bar{y})^2$$

La pureza es la medida de la homogeneidad de clase en un nodo particular; por ejemplo, el porcentaje de casos A vs casos B.

Imagina que el algoritmo está a punto de crear una rama. Tiene un grupo de datos en un nodo. Primero, calcula la impureza (digamos, el Índice de Gini) de ese grupo completo. Si el grupo está mezclado (ej. 50% transacciones fraudulentas y 50% legítimas), la impureza será alta.

“Random” del Random Forest: En lugar de evaluar todas las variables disponibles (edad, monto, ubicación, etc.), el algoritmo selecciona un subconjunto aleatorio de variables. Ejemplo: Si evalúa la variable “Monto”, probará cortes como: ¿Monto < \$100?, ¿Monto < \$500?, ¿Monto < \$1000?

Por cada corte simulado, los datos se dividen temporalmente en dos grupos (Nodo Izquierdo y Nodo Derecho). El algoritmo aplica la fórmula matemática elegida (Gini o Entropía) a cada uno de estos nuevos grupos para ver qué tan “puros” quedaron.

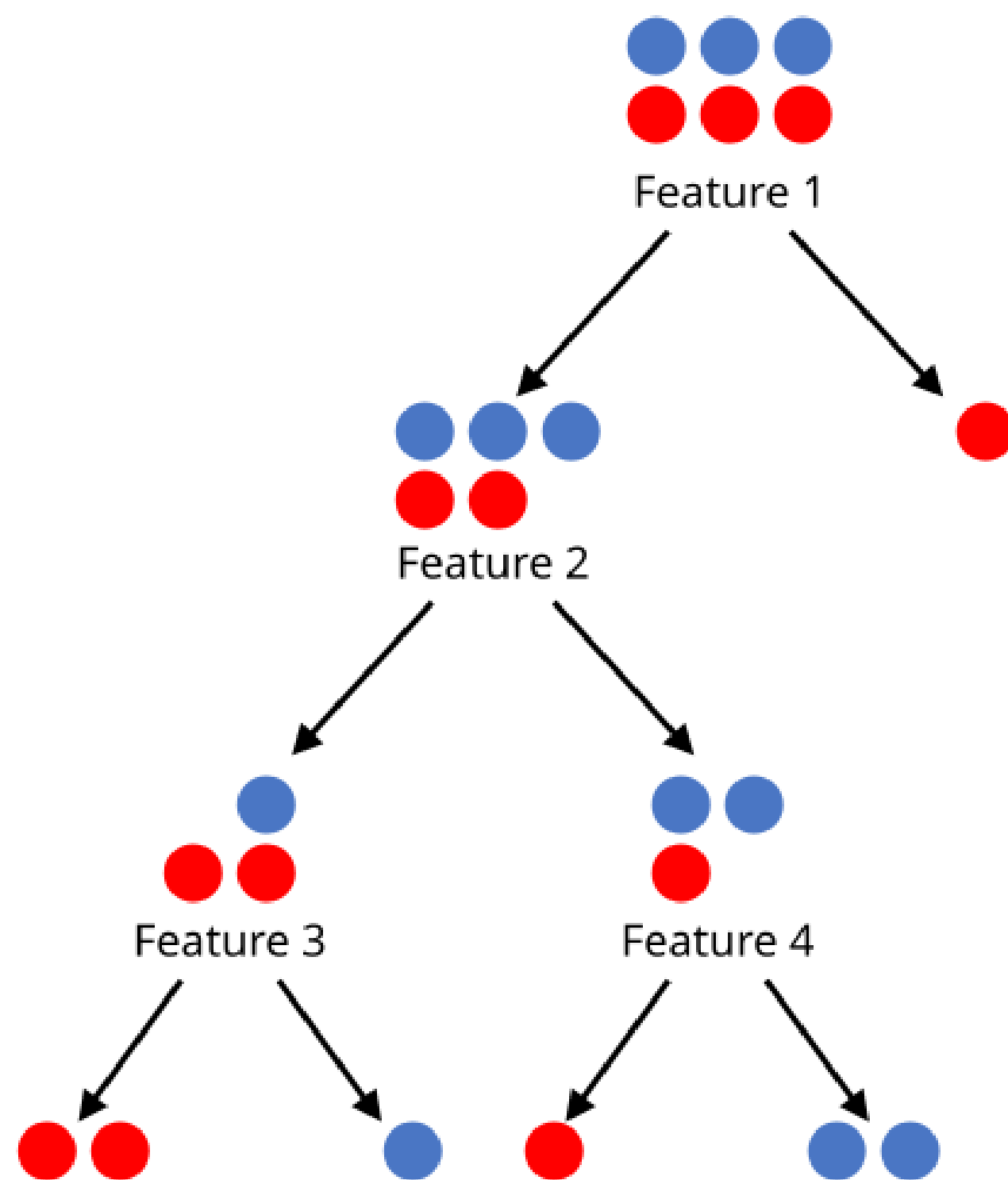
El algoritmo no se fija solo en un lado, sino en el resultado global del corte. Calcula el Gini Ponderado de ambos nodos hijos (dándole más peso al nodo que tenga más cantidad de datos). Luego, resta este resultado de la impureza que tenía el Nodo Padre originalmente. El resultado de esa resta es la **Ganancia de Información**.

Después de evaluar cientos de cortes posibles en ese subconjunto de variables, el algoritmo ejecuta la división real utilizando la variable y el punto de corte exacto que haya logrado la **mayor ganancia de información**.

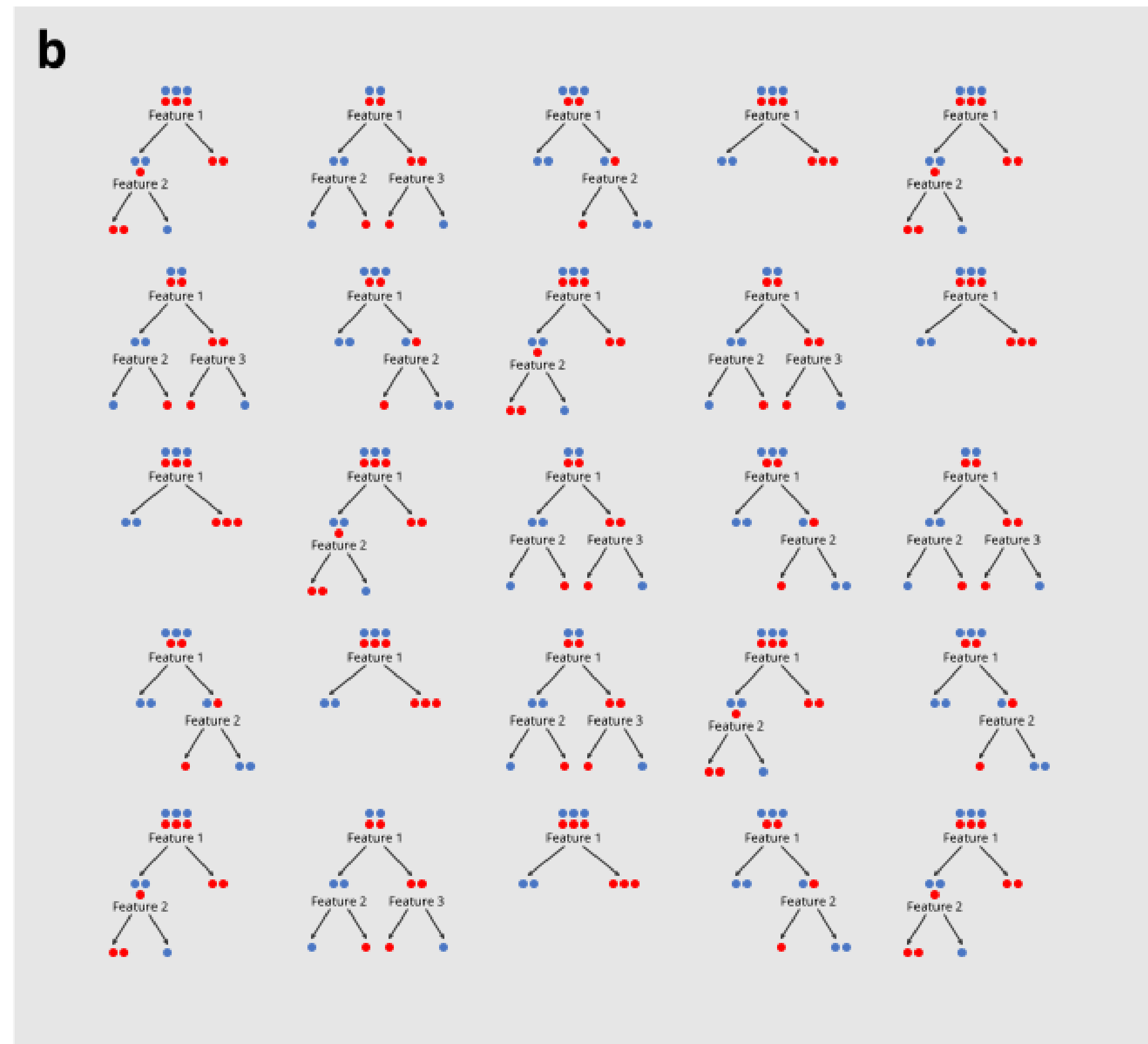
Regresión y Clasificación: CART vs RandomForest

RF (Random Forest o Bosque Aleatorio) es simplemente una colección de árboles CART (de ahí el nombre de “bosque”) que se construyen a partir de muestras bootstrap del conjunto de datos original. La clasificación se determina mediante una votación por mayoría entre los árboles del bosque.

a



b



La división cesa cuando se alcanza un umbral de pureza predeterminado; los nodos que no se dividen más se denominan nodos terminales. Los sujetos en los nodos terminales se contabilizan y, a continuación, una mayoría simple determina la clasificación del nodo.

Regresión y Clasificación: Otros tipos de árboles

Además del RandomForest, scikit-learn tiene un ecosistema muy robusto de algoritmos basados en árboles para problemas de regresión. Se dividen principalmente en tres categorías: el árbol base, y los ensambles (Bagging y Boosting)

Modelo	Valor	Cuándo elegirlo
DecisionTreeRegressor: Es la contraparte directa del clasificador que acabamos de discutir. En lugar de usar la impureza de Gini o la Entropía, decide cómo dividir los datos buscando minimizar el error (generalmente el MSE o el MAE).	Es rápido e interpretable, pero muy propenso al sobreajuste si se usa de forma individual sin limitar su profundidad.	Cuando necesitas explicabilidad total del negocio y poder graficar el 100% de la lógica.
ExtraTreesRegressor: Es una variación directa del Random Forest. Mientras que el Random Forest busca el mejor punto de corte matemático para cada variable, <i>Extra Trees</i> genera puntos de corte al azar y se queda con el mejor de esos umbrales aleatorios.	Es más rápido de entrenar que un Random Forest y a menudo generaliza mejor, ya que introduce un nivel extra de aleatoriedad que ayuda a evitar la memorización de los datos.	Cuando tu Random Forest sigue sufriendo de sobreajuste o necesitas optimizar los tiempos de entrenamiento.
HistGradientBoostingRegressor: En lugar de ordenar y evaluar cada punto de datos exacto para hacer un corte, agrupa los valores continuos en contenedores o “histogramas”.	Es exponencialmente más rápido en conjuntos de datos grandes (más de 10,000 muestras) y, a diferencia de los demás modelos estándar de la librería, soporta valores nulos (NaN) nativamente en el código sin que el algoritmo se rompa	Cuando tienes un dataset mediano/grande, valores incompletos, y necesitas exprimir el máximo rendimiento predictivo posible.
GradientBoostingRegressor: El enfoque clásico. Entrena árboles secuencialmente para predecir los “residuos” (la diferencia entre el valor real y la predicción actual).	Tremendamente preciso y suele dar mejores resultados finales que el Random Forest.	Para datasets pequeños o medianos donde se busca superar la precisión de un Random Forest tradicional.

Laboratorio 1: Regresión Lineal

- Regresión lineal al desnudo (+18)

Laboratorio 2: Regresión lineal y Random Forest

- California Housing
- EXTRA: Pesos y ONNX para producción.



Laboratorio 3: Árboles

- Clasificación y Regresión

Laboratorio 4: Tree Plot

- Decision Tree
- Random Forest



Regresión y Clasificación: Perceptron multicapa (MLP)

El perceptrón multicapa clásico, tal como fue introducido por Rumelhart, **Hinton** y Williams, se puede describir mediante:

- Una función lineal que agrega los valores de entrada.
- Una función sigmoide, también conocida como función de activación.
- Una función de umbral para el proceso de clasificación, y una función identidad para problemas de regresión.
- Una función de pérdida o costo que calcula el error global de la red.
- Un procedimiento de aprendizaje para ajustar los pesos de la red, es decir, el llamado algoritmo de retropropagación.

Vector

Un vector es una colección de números ordenados o escalares. Si estás familiarizado con el análisis de datos, un vector es como una columna o fila en un dataframe. Un vector es como un arreglo (array) o una lista.

Convencionalmente los vectores se representan como vectores columna

Minúscula en negrita indica un vector

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \dots \\ x_n \end{bmatrix} = \begin{pmatrix} x_n \end{pmatrix} \in \mathbb{R}^n$$

Índice para el caso n

Todos los elementos de x hasta n

Pertenece a

Espacio de coordenadas reales de n dimensiones

Matriz

Una matriz es una colección de vectores o listas de números. En el análisis de datos, esto es equivalente a un dataframe de 2 dimensiones. En programación es equivalente a un arreglo multidimensional (array) o una lista de listas.

Mayúscula indica una matriz $\longrightarrow$

Convencionalmente el primer índice representa filas y el segundo índice representa columna

n columnas $\downarrow$

$$\mathbf{W} = \begin{bmatrix} W_{11} & W_{12} & \dots & W_{1n} \\ W_{21} & W_{22} & \dots & W_{2n} \\ \dots & \dots & \dots & \dots \\ W_{m1} & W_{m2} & \dots & W_{mn} \end{bmatrix}$$

m filas $\uparrow$

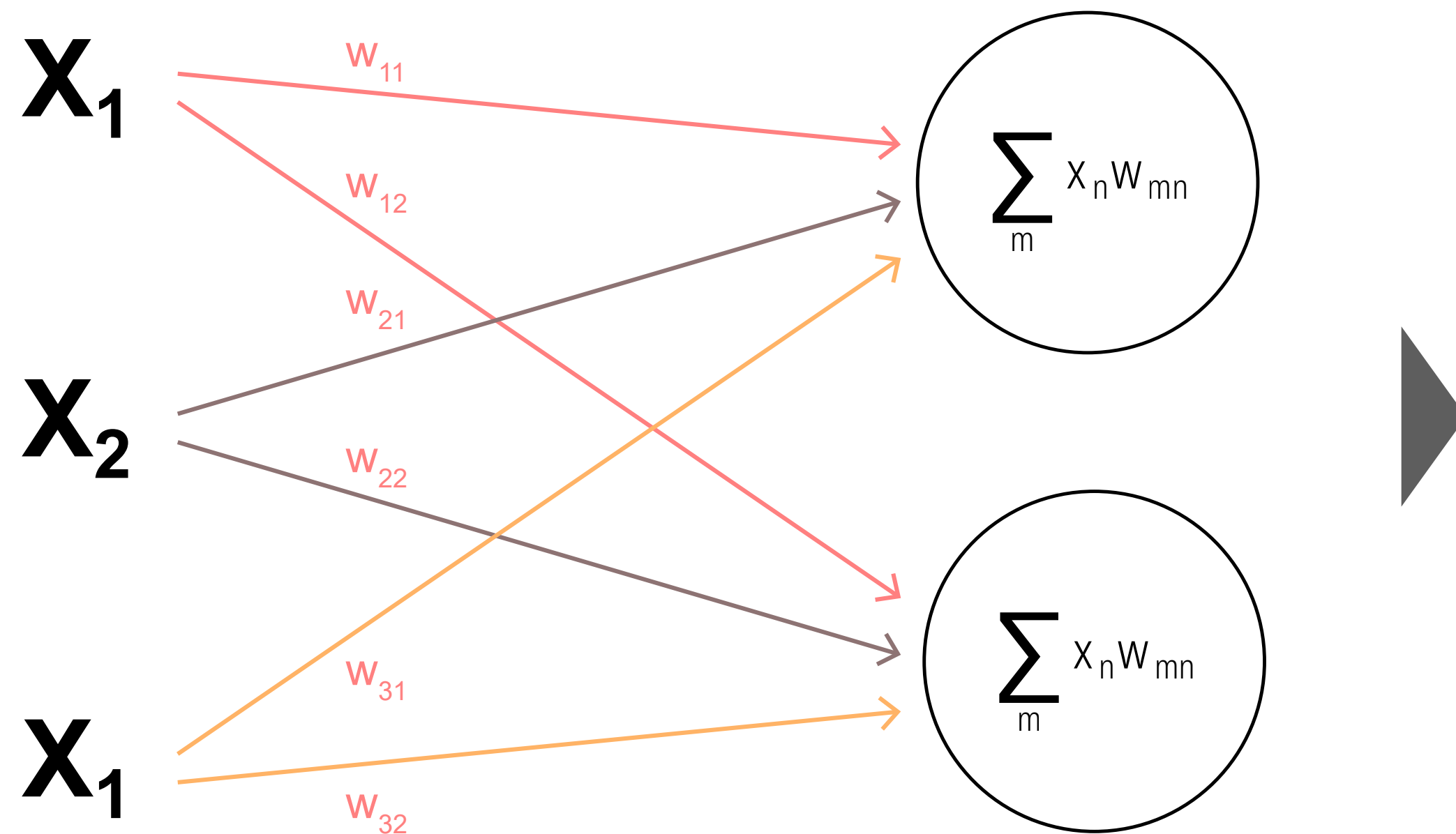
Espacio de coordenadas reales de $m \times n$ dimensiones $\downarrow$

$$= (W_{mn}) \in \mathbb{R}^{m \times n}$$

Todos los elementos de W hasta mn $\uparrow$

Perceptrón multicapa

Una matriz es una colección de vectores o listas de números. En el análisis de datos, esto es equivalente a un dataframe de 2 dimensiones. En programación es equivalente a un arreglo multidimensional (array) o una lista de listas.



$$\mathbf{x} = \begin{bmatrix} X_1 \\ X_2 \\ X_3 \end{bmatrix}$$

El vector de entrada para el entrenamiento.

$$\mathbf{W} = \begin{bmatrix} W_{11} & W_{12} \\ W_{21} & W_{22} \\ W_{31} & W_{32} \end{bmatrix}$$

Dado que tenemos 3 unidades de entrada conectadas a 2 unidades ocultas, tenemos 3x2 pesos.

$$\mathbf{z} = \mathbf{W}^T \times \mathbf{x} = \begin{bmatrix} W_{11} & W_{12} & W_{13} \\ W_{21} & W_{22} & W_{23} \end{bmatrix} \begin{bmatrix} X_1 \\ X_2 \\ X_3 \end{bmatrix} = \begin{bmatrix} Z_1 \\ Z_2 \end{bmatrix}$$

$$\mathbf{z} = \mathbf{W}^T \times \mathbf{x} = \begin{bmatrix} X_1 W_{11} + X_2 W_{12} + X_3 W_{13} \\ X_1 W_{21} + X_2 W_{22} + X_3 W_{23} \end{bmatrix} = \begin{bmatrix} Z_1 \\ Z_2 \end{bmatrix}$$

La salida de la función lineal es igual a la multiplicación del vector $\mathbf{x}$ y la matriz $\mathbf{W}$. Para realizar la multiplicación en este caso, necesitamos transponer la matriz $\mathbf{W}$ para hacer coincidir el número de columnas en $\mathbf{W}$ con el número de filas en $\mathbf{x}$. Transponer significa "voltear" las columnas de $\mathbf{W}$ de tal manera que la primera columna se convierta en la primera fila, la segunda columna se convierta en la segunda fila, y así sucesivamente.

Perceptrón multicapa

Salida de la función

Entradas de la función

Término de sesgo (bias)

Elemento n del vector de entradas

Elemento de la matriz de parámetros

$$z_m = \mathbf{f}(x_n, w_{mn}) = b + \sum_m x_n w_{mn}$$

Nombre de la función

Índice para cada neurona y fila de la matriz

Detailed description: The diagram shows the equation $z_m = \mathbf{f}(x_n, w_{mn}) = b + \sum_m x_n w_{mn}$ with various annotations. An arrow points from 'Salida de la función' to z_m . A bracket above x_n, w_{mn} is labeled 'Entradas de la función'. An arrow points from 'Término de sesgo (bias)' to b . An arrow points from 'Elemento n del vector de entradas' to x_n . An arrow points from 'Elemento de la matriz de parámetros' to w_{mn} . An arrow points from 'Nombre de la función' to $\mathbf{f}$. An arrow points from 'Índice para cada neurona y fila de la matriz' to the summation index m .

Aquí, f es una función de cada elemento del vector x y de cada elemento de la matriz W . El índice m identifica las filas en W^T y las filas en z . El índice n indica las columnas en W^T y las filas en x . Nota que agregamos un término de sesgo b , que tiene la función de simplificar el aprendizaje de un umbral adecuado para la función. Si tienes curiosidad al respecto, lee la sección "Función de agregación lineal" aquí. En resumen, la función lineal es una suma ponderada de las entradas más un sesgo.

Función de activación: Sigmoide

Cada elemento del vector z se convierte en una entrada para la función sigmoide.

Salida de la función $\rightarrow a_m = \sigma(z_m) = \frac{1}{1 + e^{-z_m}}$

Salida de la función lineal se convierte en la entrada de sigmoide

Símbolo de función sigmoide (sigma)

Símbolo de número de Euler (~ 2.71828)

La salida de sigmoide es otro vector a de m dimensiones, una entrada por cada unidad en la capa oculta, como sigue:

$$\mathbf{a} = \begin{bmatrix} a_1 \\ a_2 \end{bmatrix}$$

Aquí, $\mathbf{a}$ significa "activación", que es una forma común de referirse a la salida de las unidades ocultas. Esta función sigmoide que "envuelve" el resultado de la función lineal se denomina comúnmente función de activación. La idea es que una unidad se "activa" más o menos de la misma manera que una neurona se activa cuando recibe una señal de entrada lo suficientemente fuerte. La elección de una sigmoide es arbitraria. Se podrían seleccionar muchas funciones no lineales diferentes en esta etapa de la red, como una **Tanh** o una **ReLU**. Desafortunadamente, no existe un método estricto y fundamentado para elegir las funciones de activación de las capas ocultas. Es principalmente una cuestión de ensayo y error.

Mini-Lab

Veamos la función sigmoide.



```
from scipy.special import expit
import numpy as np
import altair as alt
import pandas as pd
z = np.arange(-5.0, 5.0, 0.1)
a = expit(z)
df = pd.DataFrame({"a":a, "z":z})
df["z1"] = 0
df["a1"] = 0.5
sigmoid = alt.Chart(df).mark_line().encode(x="z", y="a")
threshold = alt.Chart(df).mark_rule(color="red").encode(x="z1", y="a1")
(sigmoid + threshold).properties(title='Chart 1')
```

Función de salida

Para los problemas de regresión (problemas que requieren un valor de salida real, como la predicción de ingresos o calificaciones de exámenes), cada unidad de salida implementa una función identidad como:

$$\hat{\mathbf{y}} = \mathbf{f}(a_m) = a_m$$

En términos simples, una función identidad devuelve el mismo valor que la entrada. No hace nada más. El punto es que la variable a ya es la salida de una función lineal, por lo tanto, es el valor que necesitamos para este tipo de problema.

$$\hat{\mathbf{y}} = a_m = \sigma(z_m) = \frac{1}{1 + e^{-z_m}}$$

Para problemas de clasificación binaria, cada unidad de salida implementa una función de umbral como:

$$\hat{\mathbf{y}} = \mathbf{f}(a_m) \begin{cases} +1 & \text{si } a > 0.5 \\ -1 & \text{de lo contrario} \end{cases}$$

Para los problemas de clasificación multiclase, podemos usar una función **softmax** como:

$$\hat{\mathbf{y}} = \sigma(\mathbf{a})_i = \frac{e^{\beta a_i}}{\sum_{j=1}^k e^{\beta z_j}}$$

Pérdida

La función de costo es la medida de la “bondad” o “maldad” (dependiendo de cómo te guste ver las cosas) del rendimiento de la red. Este puede ser un término confuso. La gente a veces la llama función objetivo, función de pérdida o función de error.

- **Función de pérdida (Loss function):** Generalmente se refiere a la medida del error para un solo caso de entrenamiento.
- **Función de costo (Cost function):** Se refiere al error agregado para todo el conjunto de datos.
- **Función objetivo (Objective function):** Es un término más genérico que se refiere a cualquier medida del error global en una red.

En su trabajo original, Rumelhart, Hinton y Williams utilizaron la suma de errores al cuadrado definida como:

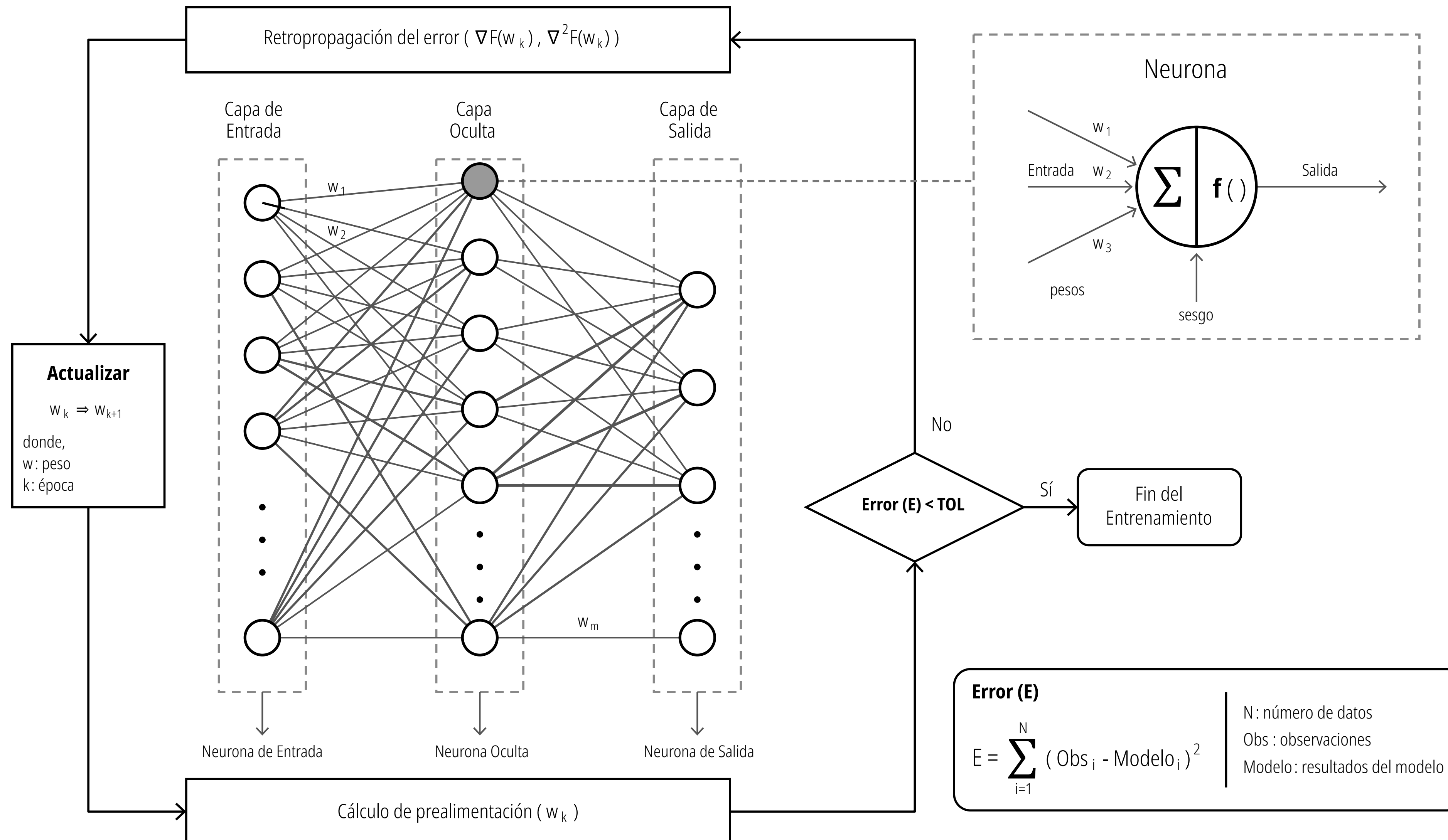
$$E = \frac{1}{2} \sum_k (a_k - y_k)^2$$

Diagrama de la ecuación de la suma de errores al cuadrado:

- Añadido para simplificar el cálculo del gradiente:** Señala al factor $\frac{1}{2}$.
- Suma de errores al cuadrado para todo el conjunto de datos:** Señala al símbolo de suma $\sum$.
- Índice para el par entrada / salida:** Señala al índice k debajo de la suma.
- Valor predicho para el ejemplo de entrenamiento k :** Señala al término a_k .
- Valor esperado para el ejemplo de entrenamiento k (ground truth):** Señala al término y_k .

Por ejemplo, el “error cuadrático medio”, la “suma de errores al cuadrado” y la “entropía cruzada binaria” son todas funciones objetivo. Para nuestros propósitos, usaré todos estos términos indistintamente: todos se refieren a la medida del rendimiento de la red. Hoy en día, probablemente desearías usar diferentes funciones de costo para diferentes tipos de problemas.

Todas las redes neuronales se pueden dividir en dos partes: una fase de propagación hacia adelante, donde la información “fluye” hacia adelante para calcular las predicciones y el error; y la fase de propagación hacia atrás, donde el algoritmo de retropropagación calcula las derivadas del error y actualiza los pesos de la red.



Propagación hacia adelante (forward propagation)

Multiplicación de matrices
De la capa de entrada hacia la capa oculta

$$\begin{bmatrix} W_{11} & W_{12} & W_{13} \\ W_{21} & W_{22} & W_{23} \end{bmatrix}^T \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

Función lineal

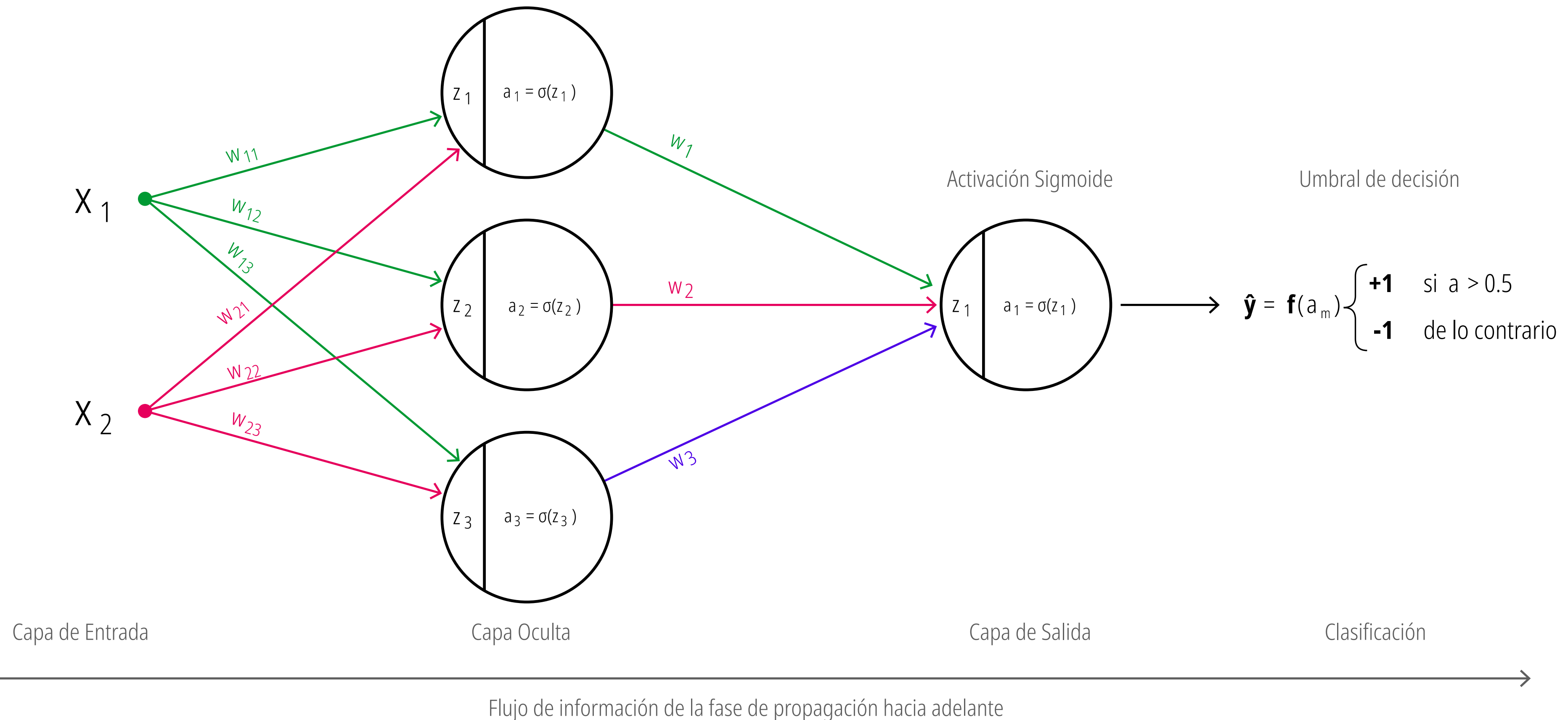
$$z_m = b + \sum_m x_n W_{mn}$$

Activación sigmoide

$$a_m = \sigma(z_m) = \frac{1}{1 + e^{-z_m}}$$

Multiplicación de matrices
De la capa oculta hacia la capa de salida

$$\begin{bmatrix} w_1 & w_2 & w_3 \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix}$$



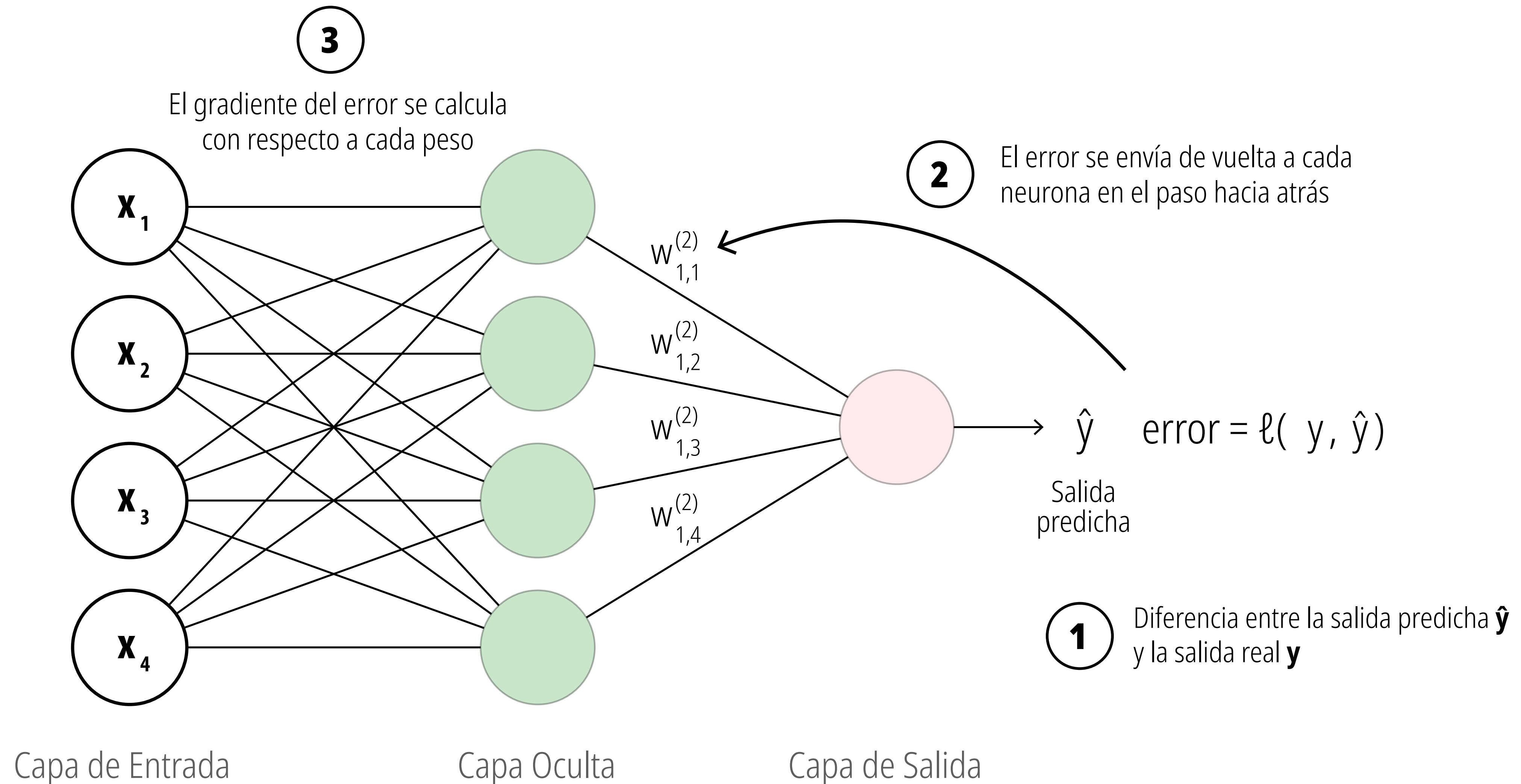
Propagación hacia adelante (forward propagation)

La fase de propagación hacia adelante implica "encadenar" todos los pasos que hemos definido hasta ahora: la función lineal, la función sigmoide y la función de umbral.

$$\hat{\mathbf{y}} = \boldsymbol{\tau} \left(\boldsymbol{\sigma}^{(3)} \left(\boldsymbol{\lambda}^{(3)} \left(\boldsymbol{\sigma}^{(2)} \left(\boldsymbol{\lambda}^{(2)} \left(\boldsymbol{\sigma}^{(1)} \left(\boldsymbol{\lambda}^{(1)} \left(\mathbf{x}_n, \mathbf{W}_{mn} \right) \right) \right) \right) \right) \right) \right) \right)$$

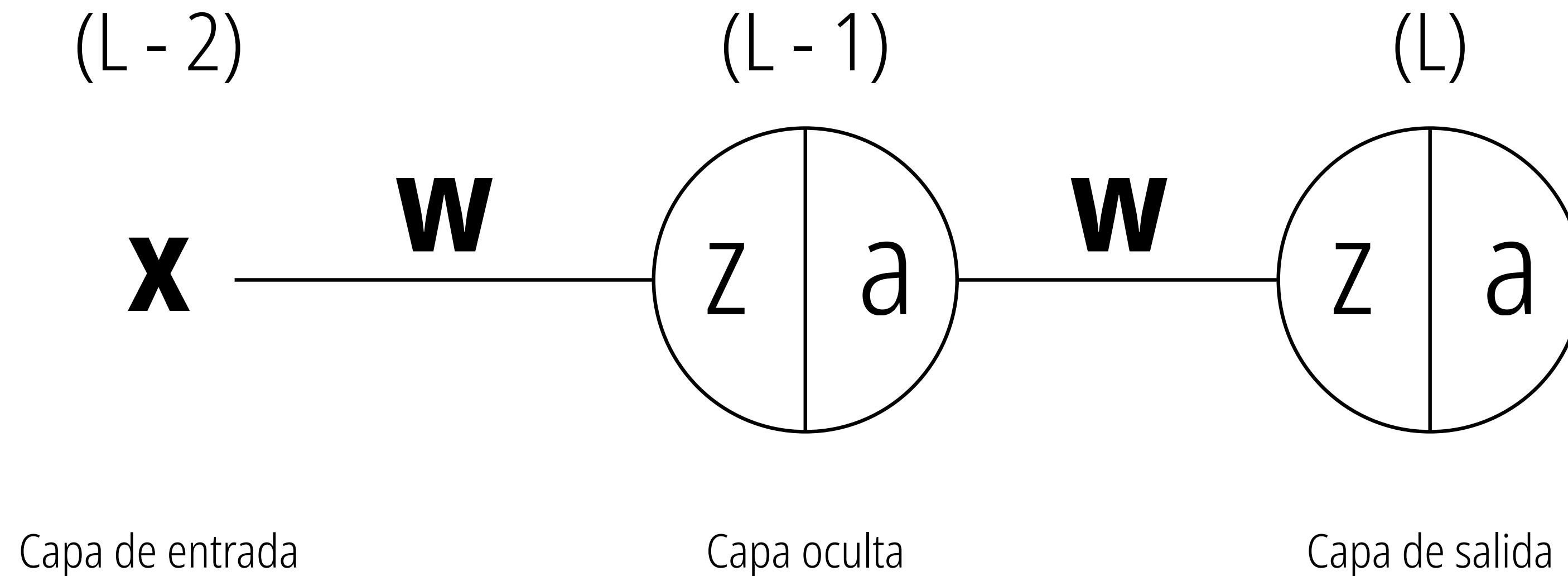
Propagación hacia atrás (backpropagation)

La fase de propagación hacia adelante implica “encadenar” todos los pasos que hemos definido hasta ahora: la función lineal, la función sigmoide y la función de umbral.



Propagación hacia atrás (backpropagation)

Se puede pensar en esto como tener una red con una sola unidad de entrada, una sola unidad oculta y una sola unidad de salida. Primero deduciremos la retropropagación para esta red simplificada y luego la generalizaremos para el caso de múltiples neuronas.



El propósito principal de la retropropagación es responder a la siguiente pregunta: “¿Cómo cambia el error cuando cambiamos los pesos en una cantidad diminuta?” (tenga en cuenta que usaré las palabras “derivadas” y “gradientes” indistintamente).

Propagación hacia atrás (backpropagation)

Para lograr esto, es necesario comprender lo siguiente:

- El error E depende del valor de la función de activación sigmoide a .
- El valor de la función de activación sigmoide a depende del valor de la función lineal z .
- El valor de la función lineal z depende del valor de los pesos w .

$$\left\{ \frac{\partial E}{\partial w^{(L)}} \right\} = \overbrace{\frac{\partial E}{\partial a^{(L)}}}^{\text{Derivada parcial del error con respecto a la activación sigmoide}} \times \underbrace{\frac{\partial a^{(L)}}{\partial z^{(L)}}}_{\text{Derivada parcial de la activación sigmoide con respecto a la función lineal}} \times \overbrace{\frac{\partial z^{(L)}}{\partial w^{(L)}}}^{\text{Derivada parcial de la función lineal con respecto al peso}}$$

$\uparrow$
 Índice de la capa actual

Propagación hacia atrás (backpropagation)

$$\frac{\partial E}{\partial w_{jk}^{(L)}} = \frac{\partial E}{\partial a_j^{(L)}} \times \frac{\partial a_j^{(L)}}{\partial z_j^{(L)}} \times \frac{\partial z_j^{(L)}}{\partial w_{jk}^{(L)}}$$



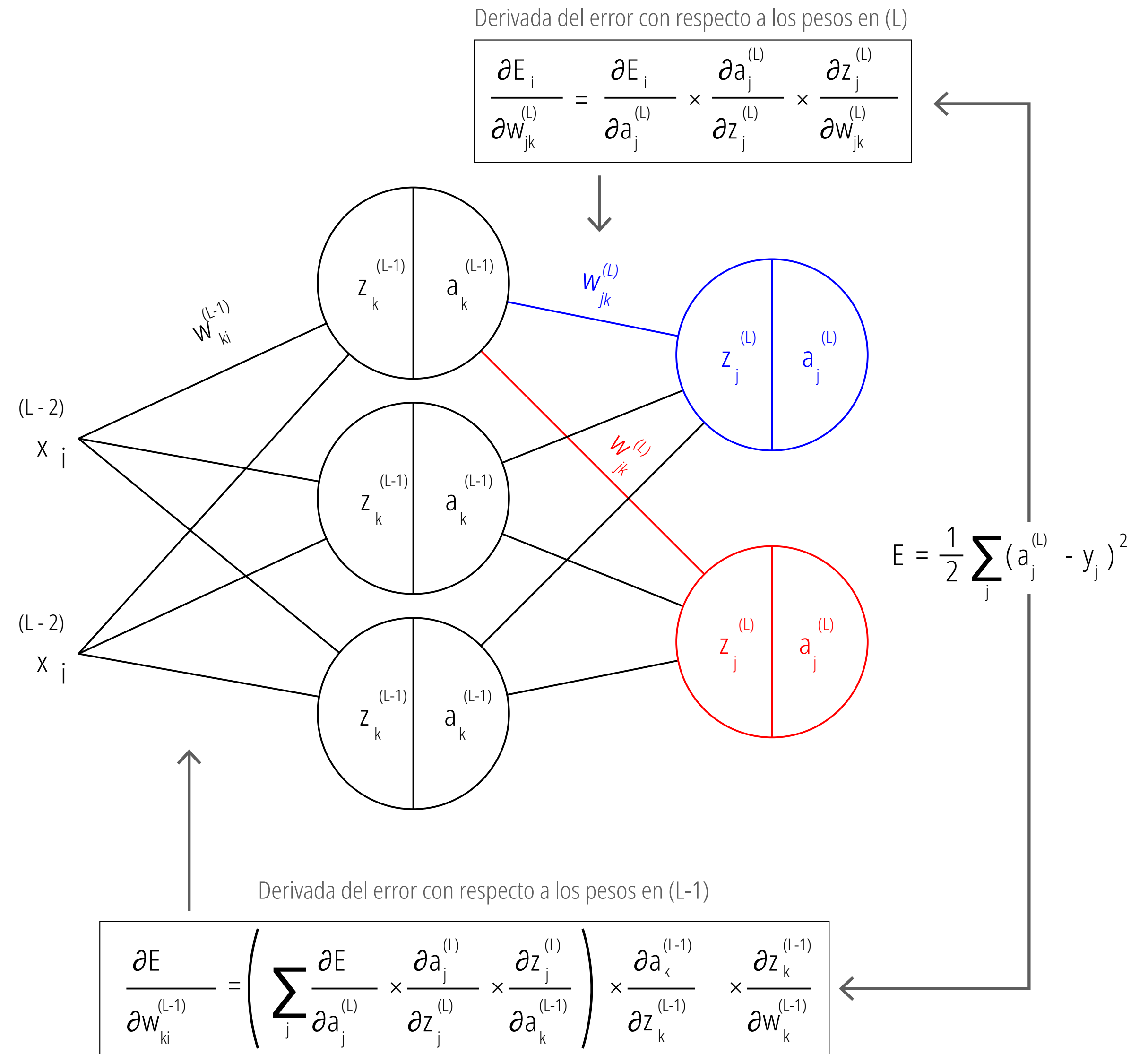
$$\frac{\partial E}{\partial w_{jk}^{(L)}} = a_j^{(L)} - y \times a_j^{(L)} (1 - a_j^{(L)}) \times a_k^{(L-1)}$$



$$\frac{\partial E}{\partial a_k^{(L-1)}} = \sum_j \frac{\partial E}{\partial a_j^{(L)}} \times \frac{\partial a_j^{(L)}}{\partial z_j^{(L)}} \times \frac{\partial z_j^{(L)}}{\partial a_k^{(L-1)}}$$

Utilizando la regla de la cadena del cálculo diferencial, el algoritmo determina exactamente qué fracción de culpa tiene cada peso y sesgo en el error total, permitiendo actualizar hasta la capa más profunda de la red.

- **Cálculo de responsabilidades (Capa de salida L):** La caja superior muestra cómo se calcula el gradiente para un peso en la última capa. La red descompone el problema multiplicando tres derivadas más simples: cómo cambia el error respecto a la activación final, cómo cambia esa activación respecto a la suma lineal (z), y cómo cambia esa suma respecto al peso específico. Es una cadena matemática de causa y efecto.
- **El efecto multiplicador en capas ocultas (Capa L-1):** La caja inferior revela la complejidad real de las redes profundas. Para saber cómo ajustar un peso en una capa oculta, el modelo ya no observa una sola salida. Como ese peso alimenta a múltiples neuronas en la capa siguiente, la matemática exige calcular y sumar los gradientes que provienen de todas las rutas a las que ese peso afectó.



Actualización de Parámetros

- **Aplicando el Descenso de Gradiente en cascada:** Las ecuaciones de la derecha son la ejecución práctica de todo el cálculo anterior. Muestran la regla de actualización estándar aplicada a diferentes profundidades de la red: restarle al valor actual del parámetro la dirección de su gradiente correspondiente.
- **Unidad de aprendizaje en toda la red:** Observa cómo la tasa de aprendizaje o tamaño de paso (η) se mantiene constante y multiplica al gradiente sin importar si estamos ajustando un peso (w) de la última capa, un peso de la capa oculta, o un término de sesgo (b). Esto asegura que toda la red avance hacia la minimización del error de forma sincronizada y controlada.

Para los pesos en la capa (L):

$$w_{jk}^L = w_{jk}^L - \eta \times \frac{\partial E}{\partial w_{jk}^L}$$

Para los pesos en la capa (L-1):

$$w_{ki}^{L-1} = w_{ki}^{L-1} - \eta \times \frac{\partial E}{\partial w_{ki}^{L-1}}$$

Para el sesgo en la capa (L):

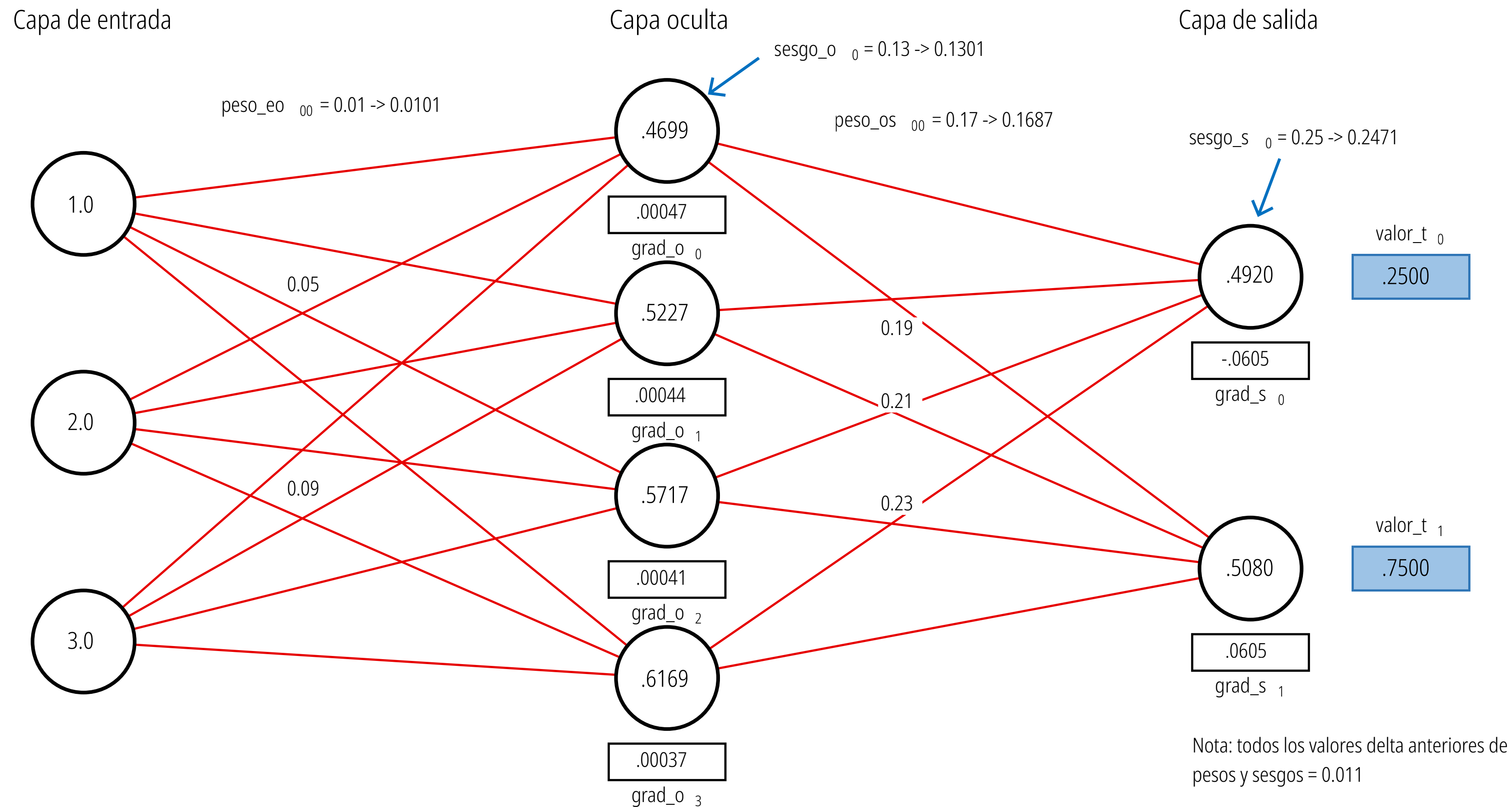
$$b^{(L)} = b^{(L)} - \eta \times \frac{\partial E}{\partial b^{(L)}}$$

Para el sesgo en la capa (L-1):

$$b^{(L-1)} = b^{(L-1)} - \eta \times \frac{\partial E}{\partial b^{(L-1)}}$$

Actualización de Parámetros

Micro-ajuste de un solo paso temporal



El Flujo Hacia Adelante (Predicción)

Arquitectura de la red: El modelo toma 3 valores de entrada (1.0, 2.0, 3.0), los procesa a través de una capa oculta de 4 neuronas, y emite 2 predicciones finales en la capa de salida.

Predicción vs. Realidad: En la extrema derecha, vemos la raíz del aprendizaje. La primera neurona de salida arrojó un valor de 0.4920, pero su objetivo real (valor_t_0) era 0.2500. La segunda arrojó 0.5080, pero debía llegar a 0.7500. Esta discrepancia matemática es lo que activa el algoritmo de corrección.

El Flujo Hacia Atrás (Retropropagación)

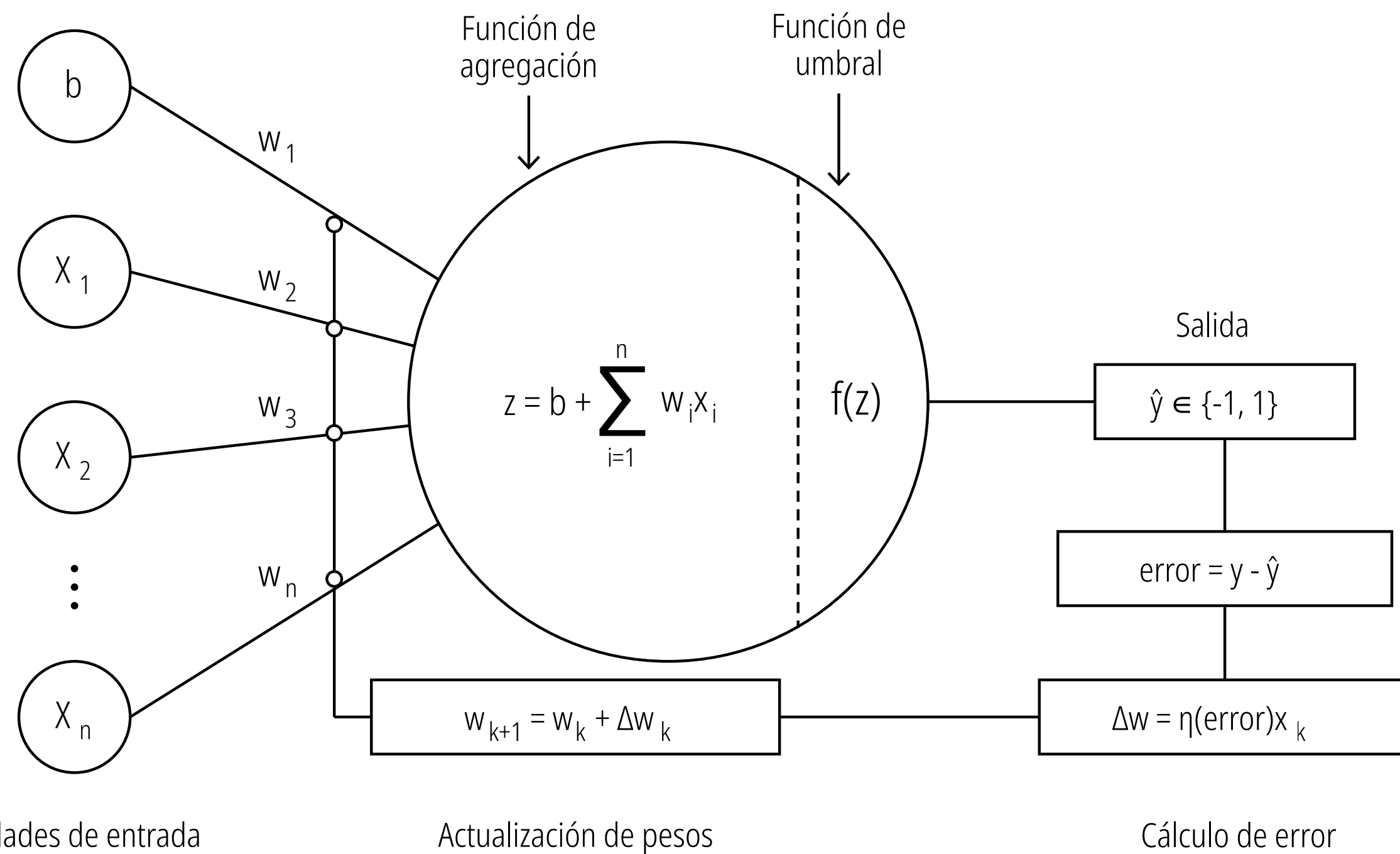
La cascada de gradientes: Los recuadros debajo de las neuronas (ej. grad_s_0 y grad_o_0) muestran la "culpa" matemática asignada a cada nodo. Observa cómo los gradientes en la capa de salida (ej. -0.0605) son más grandes, mientras que los gradientes que retroceden hacia la capa oculta (ej. 0.00047) se vuelven mucho más pequeños al diluirse por la regla de la cadena.

La actualización en acción: Las anotaciones con flechas (->) ilustran el resultado final del Descenso de Gradiente. La red toma los pesos y sesgos originales y les aplica una ligera corrección basada en esos gradientes.

Por ejemplo, un peso de salida (peso_os_{00}) se reduce de 0.17 a 0.1687. Un sesgo de salida (sesgo_s_0) baja de 0.25 a 0.2471. Un peso de entrada (peso_eo_{00}) aumenta pequeñísimamente de 0.01 a 0.0101.

El desarrollo de las redes neuronales artificiales experimentó un salto evolutivo clave en la transición del Perceptrón clásico a ADALINE. Aunque ambos modelos comparten una arquitectura idéntica de una sola capa, su diferencia radical radica en el "cuándo" y el "cómo" aprenden.

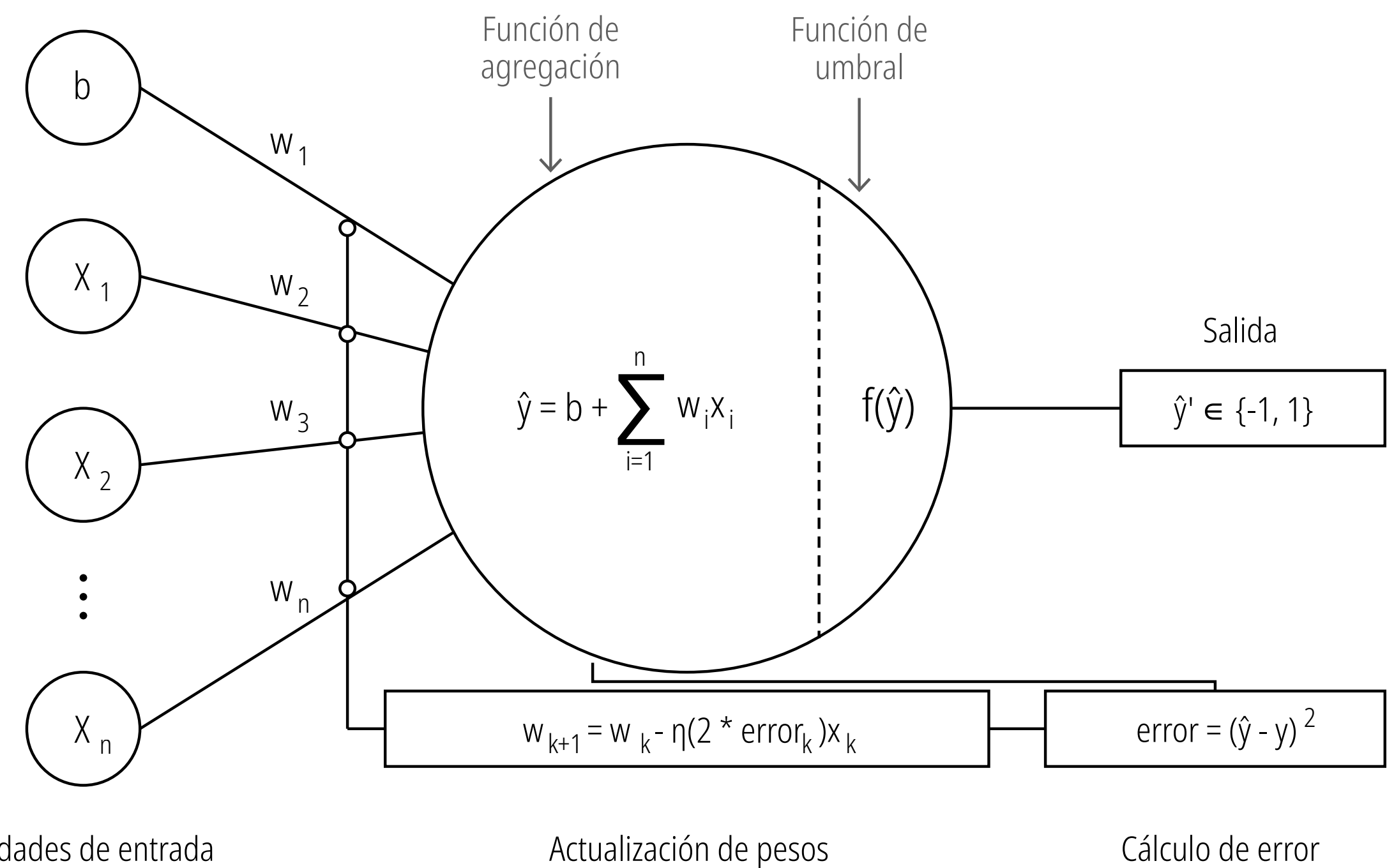
Bucle de entrenamiento del Perceptrón



Evaluación discreta (post-umbral): El modelo calcula su error comparando la etiqueta real con la salida final ya clasificada (después de pasar por la función de umbral). Es una evaluación en "blanco y negro": el algoritmo solo sabe si acertó o se equivocó, sin percibir qué tan cerca estuvo de la respuesta correcta.

Actualización reactiva y rígida: Como consecuencia de este cálculo binario, la actualización de los pesos solo ocurre cuando hay un error de clasificación. Si el modelo clasifica correctamente el dato, asume que el trabajo está terminado y no realiza ninguna mejora en sus parámetros, ignorando cualquier margen de optimización.

Bucle de entrenamiento ADALINE



Evaluación continua (pre-umbral): La gran innovación de ADALINE es que calcula el error utilizando el valor numérico puro de la función de agregación (antes de la función de umbral). Esto le permite usar una función de coste cuadrática para medir la "distancia" matemática exacta entre su predicción lineal y el resultado ideal.

Optimización proporcional (Descenso de Gradiente): Al contar con un error continuo y derivable, el modelo puede ajustar sus pesos de manera suave y proporcional a la magnitud de la falla. Esto significa que ADALINE sigue optimizando y ajustando sus parámetros para acercarse lo más posible a la perfección matemática, incluso si la clasificación final ya era correcta.

Descenso de Gradiente

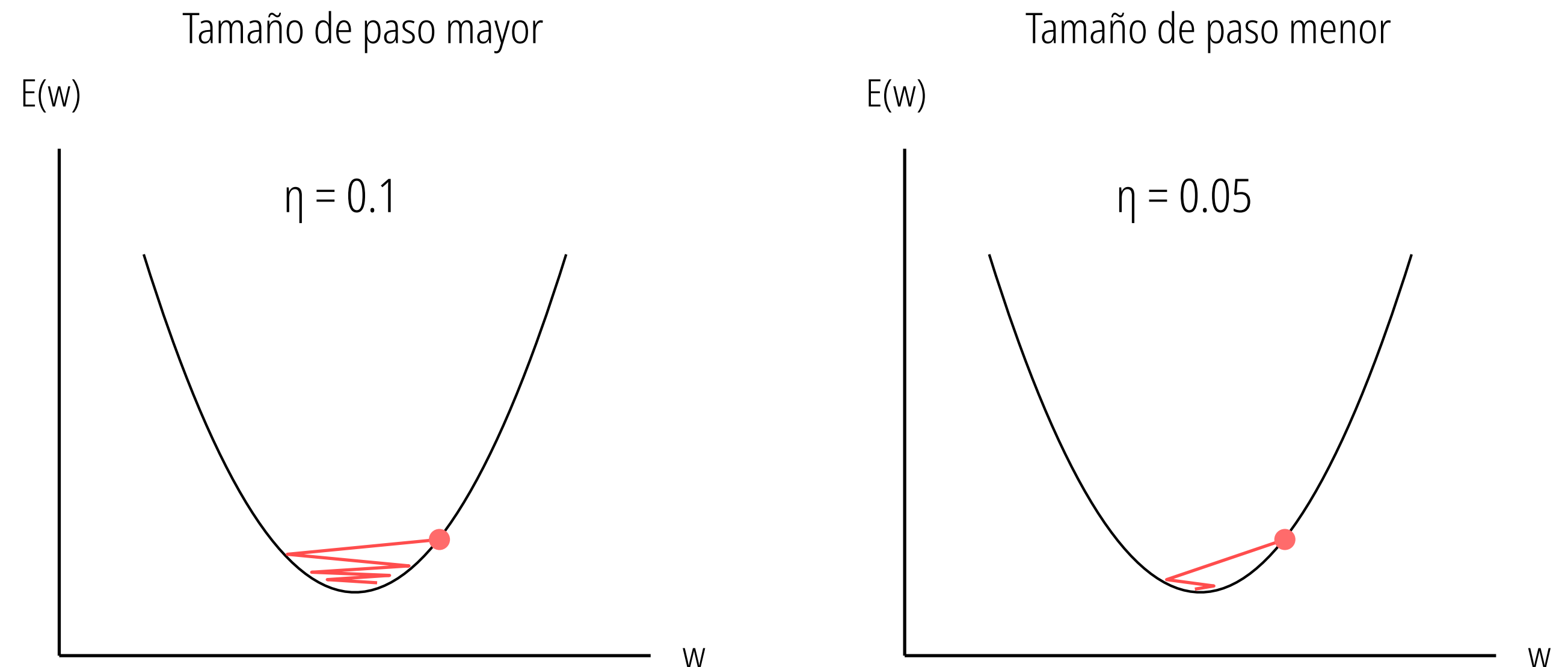
Esta técnica consiste en ajustar iterativamente los parámetros del modelo evaluando la pendiente matemática del error.

$$w_{j+1} = w_j + \eta (-\Delta_j)$$

Diagrama de la ecuación de actualización de pesos:

- w_{j+1} : peso j en el siguiente paso de tiempo (índice para cada fila de la matriz X)
- w_j : peso j en el paso de tiempo actual
- η : tasa de aprendizaje o tamaño de paso (letra griega eta)
- $-\Delta_j$: gradiente j (con signo negativo)

La ecuación describe la evolución de un peso específico. Para calcular su nuevo valor en el siguiente paso de tiempo, el algoritmo toma el peso actual y le suma la dirección del gradiente negativo. Este signo negativo es crucial, ya que obliga matemáticamente al modelo a moverse en la dirección contraria a la pendiente, asegurando un descenso hacia el punto donde el error es menor. La letra griega eta representa la **tasa de aprendizaje (o tamaño de paso)**. Actúa como un multiplicador que define qué tan grande será el ajuste aplicado al peso. Es el mecanismo de seguridad del algoritmo: sin él, la magnitud pura del gradiente podría sugerir saltos demasiado abruptos que desestabilizarían todo el proceso de entrenamiento.



Sobreajuste por exceso de velocidad (eta = 0.1): Un tamaño de paso mayor hace que el modelo salte violentamente de un lado a otro del valle de la función de error E . Al intentar descender con demasiado impulso, rebasa el punto mínimo repetidas veces ("overshooting"), provocando un patrón de zig-zag o efecto rebote que retrasa significativamente la convergencia.

Descenso controlado y eficiente (eta = 0.05): Un tamaño de paso menor y bien calibrado permite un descenso mucho más suave y directo. El modelo se desliza hacia el fondo de la curva sin rebotar drásticamente en las paredes, logrando estabilizarse en el punto de error mínimo de forma estable y segura.

Multilayer Perceptron

Tanto **MLPRegressor** como **MLPClassifier** pertenecen al módulo `sklearn.neural_network` de Scikit-Learn y se basan en la misma arquitectura matemática: el Perceptrón Multicapa (Multi-Layer Perceptron). Por esta razón, comparten casi todos sus parámetros.

¿Qué los diferencia?

Su objetivo matemático o función de pérdida. **MLPClassifier** minimiza log-loss (Entropía cruzada) para clasificación, mientras que **MLPRegressor** minimiza MSE (Error Cuadrático Medio) para regresión.

Configuración de la Arquitectura

Define la capacidad de aprendizaje de tu modelo.

Parámetro	Propósito
hidden_layer_sizes	Define profundidad y ancho (ej. (100, 50)).
activation	Función de activación (relu, tanh, logistic).

Motor de Optimización

Cómo aprende el modelo durante el entrenamiento.

Parámetro	Propósito
solver	El algoritmo utilizado para la optimización de los pesos (adam y sgd).
learning_rate	Controla el tamaño del “paso” en el ajuste de pesos.
batch_size	Tamaño del subconjunto de datos por iteración.

Multilayer Perceptron

Control y Regularización

Evita el sobreajuste (overfitting) y optimiza el tiempo.

Parámetro	Propósito
alpha	Penalización L2 (evita pesos excesivamente altos).
early_stopping	Si es True, el modelo se detiene si la pérdida no mejora.
validation_fraction	Tamaño del subconjunto de datos por iteración.
tol	Tolerancia mínima para considerar que hubo una mejora.

Buenas prácticas (Tips)

- **Reproducibilidad:** Siempre fija un random_state.
- **Escalado:** Los MLP son sensibles a la escala de los datos. ¡Usa siempre StandardScaler en tu pipeline!
- **Iteraciones:** Aumenta max_iter si recibes advertencias de "ConvergenceWarning"
- **Debug:** Usa verbose=True para visualizar el progreso de la función de pérdida durante el entrenamiento.